UNIVERSITY OF SCIENCE
ADVANCED PROGRAM IN COMPUTER SCIENCE

Nguyễn Chính Thông - Huỳnh Tuấn Minh

RESEARCH AND BUILD A SYSSECOPS SYSTEM PROCESS FOR A HYBRID CLOUD ENVIRONMENT

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

HO CHI MINH CITY, 2026

UNIVERSITY OF SCIENCE
ADVANCED PROGRAM IN COMPUTER SCIENCE

Nguyễn Chính Thông - 22125102
Huỳnh Tuấn Minh - 22125055

RESEARCH AND BUILD A SYSSECOPS SYSTEM PROCESS FOR A HYBRID CLOUD ENVIRONMENT

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

THESIS ADVISOR

PhD. Tran Trung Dung

MSc. Chung Thuy Linh

HO CHI MINH CITY, 2026

Declaration

I hereby declare that this is my own research work, conducted under the supervision of PhD. Tran Trung Dung and MSc. Chung Thuy Linh

Acknowledgements

I would like to express my sincere gratitude to my supervisor for their invaluable guidance, encouragement, and support throughout this research. Their insightful feedback and thoughtful advice have been instrumental in the completion of this thesis.

I am also grateful to the lecturers, colleagues, and friends who generously shared their knowledge and offered assistance during my studies. Finally, I would like to thank my family for their patience, understanding, and unwavering support throughout this journey.

Thesis Proposal

UNIVERSITY OF SCIENCE
ADVANCED PROGRAM IN COMPUTER SCIENCE

THESIS PROPOSAL

Thesis title:	RESEARCH AND BUILD A SYSSECOPS SYSTEM PROCESS FOR A HYBRID CLOUD ENVIRONMENT
Thesis advisor:	PhD. Tran Trung Dung, MSc. Chung Thuy Linh
Students:	Huynh Tuan Minh (22125055) – Nguyen Chinh Thong (22125102)
Type of thesis:	Technology with demo application
Duration:	30th September 2025 to 4th August 2026

Contents of thesis:

1. Introduction

Traditional software development in hybrid cloud environments usually follows a linear process: requirements, design, implementation, testing, and deployment. Although CI/CD automation has accelerated testing and release, the coding and secure design stages remain costly and slow. Recent advances in large language models (LLMs) make automatic code generation

from natural language practical, reducing development effort and delivery time.

However, AI-generated code is probabilistic and can contain logic errors, insecure patterns, dependency risks, and policy violations. These risks become more critical in hybrid cloud deployments, where workloads span public cloud and on-premises infrastructure, each with different trust boundaries, network controls, and compliance requirements.

This thesis proposes a SysSecOps workflow that combines AI-assisted code generation with layered security controls and automated operations. The workflow includes prompt governance, pre-execution validation, static and dependency analysis, risk scoring, sandbox execution, runtime monitoring, and automated incident response.

The architecture explicitly addresses:

- Cross-environment identity and access control (cloud IAM + on-prem policy).
- Data and artifact movement between cloud and on-prem zones.
- Consistent security checks across heterogeneous runtime environments.
- Failover and observability across distributed infrastructure.

2. Objectives

Primary objective:

The primary objective of this research is to design and develop a SysSecOps operational process for hybrid cloud environments that integrates system administration, continuous security, and software delivery into a unified workflow, improving the coordination between infrastructure management, security operations, and software development while supporting secure, automated, and efficient deployment across heterogeneous cloud infrastructures.

Specific objectives:

1. Investigate the operational and security challenges of hybrid cloud environments and analyze the limitations of existing approaches to integrating system administration, software development, and continuous security.

2. Design a SysSecOps operational process that integrates infrastructure management, automated security assessment, policy enforcement, and deployment into a single, continuous-security workflow.

3. Develop a prototype SysSecOps system implementing the proposed process, integrating multiple security assessment tools, infrastructure automation, and continuous security mechanisms within a hybrid cloud environment.

4. Establish a unified security assessment mechanism that aggregates findings from heterogeneous security tools – including a learned code-risk signal – and supports risk evaluation to prioritize remediation.

5. Evaluate the prototype through representative hybrid-cloud deployment scenarios, examining operational feasibility, applicability to an existing on-premises target, and the correctness of the unified risk-evaluation mechanism.

3. Scope of the Thesis

Included scope:

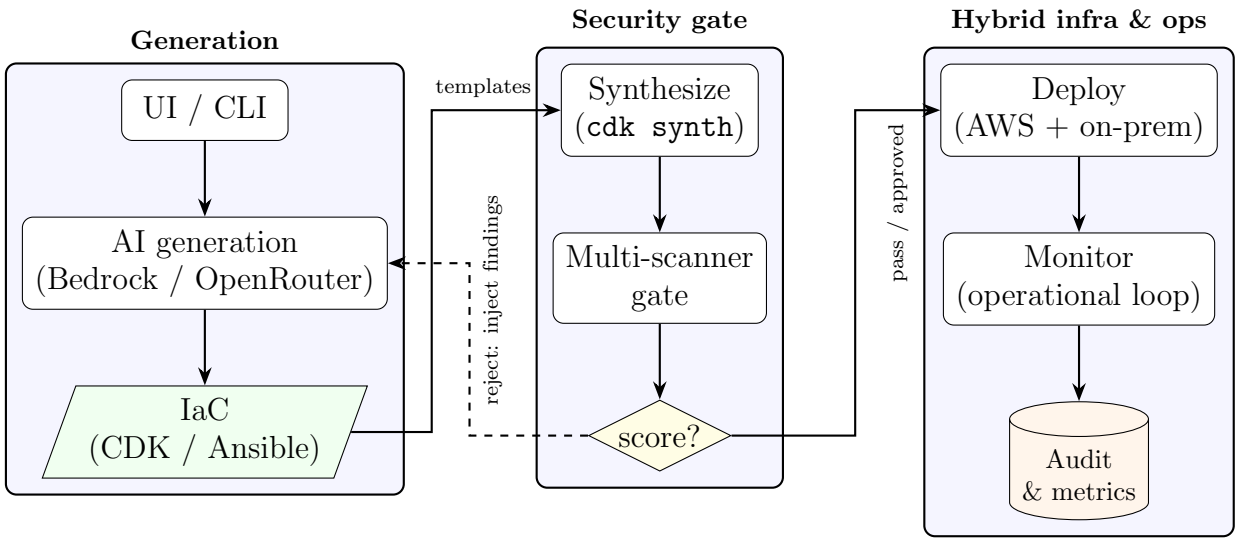
- An end-to-end prototype covering AI-assisted generation, synthesis/validation, the multi-scanner security gate, risk scoring, deployment governance, and post-deployment monitoring.

- A hybrid testbed consisting of an AWS account (public side) and on-premises Linux nodes (private side) joined by a secure mesh-VPN connection.
- Security validation of AI-generated application code and IaC artifacts.
- A learned code-risk component (logistic regression over Bandit/Semgrep features) as one input to the risk-scoring engine.

Excluded scope:

- Training large language models from scratch.
- Enterprise-scale, multi-account, or multi-tenant production deployment.
- Formal, statistically powered benchmarking against a manual-operations baseline.

4. System Architecture and Technology Stack



The system is organised into three cooperating zones. Zone 1 produces IaC; Zone 2 evaluates and gates it; Zone 3 provisions the hybrid infrastructure and continuously observes it. The risk-scoring engine sits at the boundary of Zone 1 and Zone 2, consuming scanner output and emitting the decision that governs whether code proceeds to deployment or returns to Zone 1 for regeneration.

The diagram shows the three zones and the principal flow of control between them: the solid arrows denote the forward path of a change from prompt to deployed and monitored infrastructure, and the dashed arrow denotes the remediation feedback that returns rejected generated code to Zone 1.

AI and orchestration:

- OpenAI API (LLM-based code generation)
- Python orchestration service (FastAPI)

Security pipeline:

- Bandit, Semgrep, Checkov, cfn-lint, ansible-lint, infracost
- OWASP Dependency-Check (optional secondary validation)

Execution and deployment:

- On-prem environment: VMware/VirtualBox-based Linux nodes
- Terraform/CDK for reproducible infrastructure provisioning

Monitoring and response:

- Prometheus (on-prem workloads)
- Alertmanager/webhook automation for quarantine, rollback, or block actions

5. Methods

This research adopts a design-and-evaluation methodology to develop and validate the SysSecOps workflow.

1. System Design

A modular architecture will be designed to integrate AI-based code generation with multiple security layers, including validation, analysis, and monitoring components.

2. Implementation

A prototype system will be developed using appropriate tools and technologies, such as large language model APIs for code generation, static analysis tools for vulnerability detection.

3. Security Analysis Techniques

The system will incorporate:

- Static code analysis for identifying known vulnerability patterns.
- Dependency and library scanning for supply chain risks.
- Risk scoring algorithms to prioritize potential threats.

4. Experimental Setup

The workflow is exercised through representative pass, review, reject, and degraded-component (missing scanner/credential) scenarios on the public-cloud (CDK) side, and through configuration of an existing on-premises virtual machine over a mesh-VPN connection on the private (Ansible) side.

5. Evaluation Metrics

The system will be evaluated based on:

- Operational feasibility and reliability: completion rate, stage-level completion, unexpected failures versus valid security rejections, and graceful degradation.

- Hybrid infrastructure applicability: connectivity, assessment, gate enforcement, configuration application, and state verification for an existing on-premises node.

- Unified risk-evaluation capability: accuracy, precision, recall, F1-score, and ROC–AUC of the learned code-risk component, plus correctness of score aggregation and decision thresholds.

6. Limitations

Despite the effectiveness of the proposed SysSecOps workflow in mitigating risks associated with AI-generated code, several limitations remain:
Dependence on LLM Training Data:

- The model may produce outdated practices, insecure patterns, or suboptimal designs if such patterns exist in its training corpus.

- The model may not fully capture domain-specific requirements or contextual constraints.

False Positives vs. False Negatives Trade-off:

- Static analysis and security scanning tools (e.g., Checkov, Semgrep) may generate false positives, identifying issues that are not actual vulnerabilities.

- Reducing false positives by relaxing detection rules can increase the risk of false negatives, where real vulnerabilities are missed.

System Overhead:

- Additional computational overhead, increasing build time and resource consumption compared to traditional pipelines.

Incomplete Coverage:

- Not guarantee that all execution paths are fully tested or validated, even with automated testing mechanisms.

6. Expected Results

The proposed workflow is expected to:

- Reduce secure delivery time compared with traditional development pipelines.
- Increase vulnerability detection coverage for AI-generated code and IaC.
- Reduce production risk by runtime response automation.
- Demonstrate stable and policy-consistent operation across hybrid cloud infrastructure.

Final deliverables:

1. A working SysSecOps prototype for AI-generated code.
2. A hybrid cloud experimental environment definition (CDK app, Ansible playbook).
3. Evaluation dataset and benchmark scenarios.
4. Comparative performance and security analysis report.

Research Timelines

Reference

1. R. Santos, A. Correia, and J. P. Sousa, "Security Best Practices for Cloud-Native Applications: A Systematic Review," *IEEE Access*, vol. 11, pp. 45230–45245, 2023.

Table 1: Research timeline.

Tasks	**Time**	**Duration**	**Individual**
Study AWS cloud infrastructure	9/2025	3 months	Minh, Thong
Design pipeline	1/2026	1 month	Minh
Select AI model	1/2026	1 month	Thong
Select service	3/2026	1 week	Thong
Configure service, IAM policies, secure API	15/3/2026	1 week	Thong
Draw and fix pipeline	15/3/2026	1 week	Minh, Thong
Implement pre-validation	15/3/2026	2 weeks	Minh
Bring pre-validation to service	29/3/2026	1 week	Minh, Thong
Implement security analysis	5/4/2026	2 weeks	Minh
Setup sandbox execution	5/4/2026	2 weeks	Thong
Bring security analysis to service	19/4/2026	1 week	Minh, Thong
Implement risk scoring	5/2026	1 month	Minh
Setup runtime monitoring	5/2026	1 month	Thong
Write paper	6/2026	2 months	Minh, Thong
Evaluate and finalize paper	8/2026	1 week	Minh, Thong

2. HashiCorp, "Terraform: Infrastructure as Code," HashiCorp Documentation, 2024. [Online]. Available: `https://developer.hashicorp.com/terraform/docs`.

3. OWASP Foundation, "OWASP Top 10," 2021 (vulnerability taxonomy baseline).

4. Shetti, Milan, *Nubes: Towards building a Secure and Scalable Hybrid Cloud Infrastructure* (University of Minnesota thesis, 2021).

5. Shaileshbhai, Khushi, *Security Challenges and Mitigation Strategies in Hybrid Cloud Environments* (January 31, 2025). Available at SSRN: https://ssrn.com/abstract=5118715 or http://dx.doi.org/10.2139/ssrn.5118715.

Approved by the advisor	Ho Chi Minh city, 06/04/2026
Signature of advisor	Signature(s) of student(s)

Contents

List of figures

List of tables

Abstract

Cloud computing has become a fundamental infrastructure for modern software development by providing scalable automated on-demand services. Furthermore, hybrid cloud has gained importance thanks to its characteristic including the flexibility of public-cloud services with the security of private infrastructure. However, operating these heterogeneous environments introduces challenges in maintaining consistent configurations, reconciling infrastructure changes, and enforcing security policies. Moreover, existing practices often separate system administration, software development, and security assessment into different tools and workflows, which can delay feedback and produce distinct findings. Although existing research has improved individual cloud capabilities through many approaches, specifically DevOps, and security assessment tools, less research has been given to combining these inventions into an operational process. This research aims to design and develop a SysSecOps operational process that merges infrastructure management, continuous security, and software delivery within a hybrid cloud environment. The proposed process evaluates infrastructure, aggregates and unite findings from multiple assessment sources, calculates a risk score, applies a pass, review, or reject decision for deployment,and monitoring the applications. Two practical scenarios were designed for the above challenges to analyse the proposed process in a practical operational environment. In addition, a labelled dataset containing vulnerable and reference secure code was constructed to train and evaluate the risk-scoring model to distinguish vulnerabilities from safe samples. The results indicate

that the proposed process has the potential in scalable cloud operational tasks, while the risk-scoring model can provide useful suggestions for identifying and prioritising vulnerabilities. The research provides a foundation for more consistent, traceable, and risk-aware operations for hybrid cloud infrastructure.

Chapter 1

Introduction

1.1 Growth of cloud computing

The paradigm of cloud computing presents the fact that it is fulfilling a long-held dream in the information technology industry: computing as a utility [1]. This transformation has reshaped the way IT hardware is designed, purchased, and utilized, reconstructing the industry from a large up-front capital to a pay-as-you-go economic model. Historically, the growth of cloud computing was observed by the development of large-scale commodity-computer data centers, allowing providers to achieve significant economies of scale in electricity, hardware, and operations [1].

In addition, [1] shows three new aspects that mark the initial emergence of cloud adoption. Those are the illusion of infinite computing resources available on demand, the elimination of up-front commitments by users, and the ability to pay for resources on a short-term, as-needed basis. Thus, these aspects give cloud computing elasticity, which allowed organizations to attach resources to workloads much more closely, transferring the risks of overprovisioning and underprovisioning from the service operator to the cloud vendor [1]. A notable early example of this growth was the Animoto service, which scaled from 50 to 3,500 servers in three days after it was deployed on Facebook.

Furthermore, the growth of cloud computing is characterized by a de-

centralized infrastructure through emerging architectures such as serverless computing and edge computing [2]. Serverless architectures (Function-as-a-Service) allow developers to concentrate solely on application logic while the platform manages all underlying scaling, fault tolerance, and back-end management. Simultaneously, the increase in Internet of Things (IoT) devices has driven the adoption of edge and fog computing, bringing processing power closer to data sources to meet ultra-low-latency requirements and reduce network bottlenecks [2]. This continuous expansion into distributed, heterogeneous, and ad hoc environments highlights the increasing complexity of the modern cloud ecosystem.

As technology developed, the landscape evolved from single-provider environments to more complex infrastructures [2]. Today, hybrid cloud and multi-cloud models have become the dominant standard. [2], [3] indicate that approximately 73% of organizations now employ a hybrid cloud approach, combining public and private environments to balance cost, performance, and governance. Large enterprises, in particular, have observed a significant increase in cloud spending, with more than 76% of large organizations spending more than $5 million monthly on public cloud services by 2026. However, as these hybrid cloud architectures grow in sophistication and monthly spend, they require centralized oversight through a Cloud Center of Excellence (CCOE) or FinOps teams to manage the growing complexity and associated security risks [3]. Consequently, operational and security processes are required for hybrid cloud infrastructure management.

1.2 Emergence of hybrid cloud

In recent years, hybrid cloud has been widely adopted, transforming operations in many enterprise IT. As a result, this transformation helps organizations to be capable of integrating private and public cloud infrastructures into a single computing environment [4], [5]. This deployment

model allows enterprises to optimize resource utilization, improve business continuity, and maintain diverse regulatory and operational requirements while ensuring the flexibility to successfully deploy workloads across heterogeneous infrastructures. Moreover, [5] defines a base hybrid cloud that facilitates portability and interoperability of the application between multiple cloud environments. In this way, organizations can benefit from the scalability and elasticity of public cloud services without giving up control over sensitive data and mission-critical applications.

However, the integration of multiple cloud environments significantly increases the complexity of infrastructure management [6], [7], [5]. In general, hybrid cloud models consist of heterogeneous technologies, distributed resources, and multiple security models that must operate as a cohesive system. Consequently, organizations are required to maintain consistent governance, identity and access management, and security policies in both private and public cloud in hybrid cloud. As cloud-native applications and distributed services continue to scale, ensuring operational consistency and maintaining a security standard become increasingly difficult [8].

Furthermore, hybrid cloud environments require strict coordination between infrastructure management, application deployment, and security operations [6], [8]. Traditional security approaches, which are based on periodic assessments or isolated security modules, are incapable of addressing the dynamic nature of hybrid cloud infrastructures. Instead, security controls must be integrated throughout the system lifecycle to support continuous monitoring, automated vulnerability assessment, and policy enforcement. Consequently, organizations require operational processes that are integrated with automation, infrastructure management, and continuous security practices to effectively manage hybrid cloud environments while maintaining security, and compliance [7].

1.3 Research challenges

Although hybrid cloud provides flexibility and scalability, it also introduces some significant challenges in managing secure delivery and infrastructure operations. In addition, the hybrid cloud requires organizations to reconcile software deployment, security policies, and compliance while maintaining system performance. These challenges are identified as follows.

Challenge 1: Managing heterogeneous infrastructure. Hybrid cloud refers to multiple diverse computing platforms, including on-premises infrastructure, private cloud, and public cloud, where each one has different architectures, interfaces, and security models. Consequently, the transition from the traditional cloud model to the hybrid cloud model requires maintaining consistent operational procedures and configuration policies, which is considered to be complex and time-consuming. [5] also states that the hybrid cloud needs interoperability and portability among many distinct cloud infrastructures while preserving their individual characteristics. As a result, operational teams must manually manage diverse infrastructures that often consist of different methods, mechanisms, and security policies.

Moreover, [7], [8] agree that as organizations expand their models, the complexity of models escalates, increasing administrative overhead and the likelihood of security misconfigurations. This, by far, makes consistency across heterogeneous infrastructure increasingly difficult. [8] highlights that the increasing complexity in cloud infrastructure has become one of the primary concerns for most organizations. In addition, differences in configuration policies, identity management, and resource allocation increase administrative overhead and configuration inconsistencies that may expose security vulnerabilities.

Challenge 2: Integrating continuous security into the software lifecycle. Traditional security operations are usually performed during the later stages of software development, making vulnerabilities more expensive

and difficult to detect and remediate. Although this approach is capable of maintaining security standards for conventional software development models, it may expose serious vulnerabilities in modern cloud-native applications, which are continuously developed and deployed through automated CI/CD pipelines. Thus, [9] emphasizes that security operations should be continuously incorporated into software development and deployment processes to reduce software vulnerabilities and improve software integrity.

In hybrid cloud infrastructure, applications can be deployed across multiple infrastructures with different security requirements. Therefore, Security operations must perform not only on application source code but also on software dependencies, configuration policies, and resource allocation. Performing these assessments independently or only before production deployment increases the risk that vulnerabilities remain undetected until later stages, where remediation is more expensive and difficult.

Furthermore, organizations must integrate automated security operations into existing development workflows without interfering with development productivity. Additionally, Continuous vulnerability scanning and policy validation should operate alongside continuous integration and deployment processes while maintaining acceptable execution time. Consequently, many organizations try to balance between rapid software delivery and continuous security in hybrid cloud infrastructure [9].

Challenge 3: Assessing security findings and prioritizing risks. Modern software applications are usually developed with multiple security evaluation tools. These include static application security testing, software composition analysis, dependency vulnerability scanners, and infrastructure configuration analysis tools. Each tool focuses on detecting different categories of vulnerabilities and produces systematic reports with its own severity metrics and confidence levels. Although these tools cover each security category, they also generate diverse security findings that are dif-

ficult to evaluate consistently and prioritize risks.

In practice, security engineers manually analyze findings from multiple scanners, eliminate duplicate reports, and compare vulnerabilities to decide multiple actions for each risk. However, the manual process becomes more challenging as project size grows, which can slow down the whole deployment. In hybrid cloud infrastructures, where applications are updated and deployed frequently on multiple clouds, delayed or inconsistent prioritization may leave critical vulnerabilities unresolved while less significant issues receive unnecessary attention.

[9] recommends integrating security operations into a software development process to support continuous vulnerability management and prioritization. Nevertheless, effectively combining many different security findings with distinct severity metrics into meaningful risk assessments remains a significant challenge. Thus, organizations require systematic approaches capable of combining multiple security reports and prioritizing vulnerabilities according to their overall operational risk.

Challenge 4:Establishing an integrated SysSecOps process. The complexity of hybrid cloud environments requires close contact between infrastructure management, software delivery, and security operations. However, these activities are performed by separate teams using independent tools and workflows. Specifically, infrastructure administrators concentrate primarily on provisioning and maintaining computing resources, development teams emphasize rapid software delivery, while security teams can only perform security assessments independently after implementation. Consequently, these different processes create communication gaps, which delay security reports and reduce the overall efficiency of software delivery processes.

Although DevOps has significantly reduced the communication gaps between development and security teams, security assessments remain inconsistent across many organizations. Moreover, security operations are usu-

ally known as isolated scanning stages rather than as continuous processes from infrastructure management, software development, deployment, to maintenance. As a consequence, when hybrid cloud infrastructures continue to grow in scale and complexity, security operations become increasingly difficult to manage while maintaining consistent security policies and regulatory compliance [8], [5].

1.4 Research gaps

Multiple literature reviews report that the current research still focus on particular modules across several dimensions rather than a continuos operations. [10] showed that DevSecOps research mainly concentrates on practices and tools rather than comprehensive operational frameworks. Similarly, [11] identified continuous security integration, automation, and security orchestration as persistent research challenges through a systematic review of DevSecOps studies. More recently, [12] concluded that opportunities remain for broader operational integration and more comprehensive validation of DevSecOps approaches across complex computing environments. These observations suggest that several important research gaps exist, and are as follow.

Gap 1: Limited Operational Frameworks for Hybrid Cloud Security. Existing research has proposed numerous DevSecOps practices, cloud security frameworks, and automated security solutions to improve secure software delivery. However, these studies primarily focus on software development activities, security tools, or individual operational practices instead of providing an integrated operational process capable of coordinating infrastructure management, software deployment, and continuous security within hybrid cloud environments. While the NIST Cloud Computing Reference Architecture defines the architectural relationships among cloud components and describes the characteristics of hybrid cloud infrastructures, it

does not prescribe an operational process for integrating system administration, software engineering, and security activities [5]. Likewise, [10], [12] emphasized that existing DevSecOps research is largely organized around practices, tools, and lifecycle activities, whereas comprehensive operational frameworks remain comparatively limited.

Gap 2: Insufficient Integration of Continuous Security Throughout Hybrid Cloud Operations. Continuous security has become a fundamental principle of secure software engineering, and the NIST Secure Software Development Framework recommends integrating security practices throughout software development and deployment activities to improve software integrity and reduce vulnerabilities [9]. Nevertheless, implementing continuous security remains a persistent challenge in practice. [11] identified automation of security activities, continuous security assessment, and balancing software delivery speed with security assurance as recurring challenges across existing DevSecOps studies. Similarly, Myrbakken and Colomo-Palacios reported that security integration across the software lifecycle remains one of the central themes requiring further investigation [10]. While current research mainly concentrates on integrating security into CI/CD pipelines, comparatively fewer studies investigate how continuous security should be coordinated across both software engineering activities and infrastructure operations in heterogeneous hybrid cloud environments.

Gap 3: Limited Orchestration of Heterogeneous Security Tools and Security Findings. Modern software projects increasingly employ multiple security assessment tools, including static application security testing, software composition analysis, dependency vulnerability scanning, and infrastructure security analysis. Although these tools improve vulnerability detection from different perspectives, they typically operate independently and generate heterogeneous outputs with different reporting formats, severity classifications, and confidence levels. Zhao et al. observed that existing DevSecOps literature primarily categorizes research according to security

practices, tools, and metrics, while comparatively less attention has been devoted to the orchestration and integration of heterogeneous security tools into unified operational workflows [12]. Rajapakse et al. likewise identified tool integration and security automation as recurring research challenges affecting DevSecOps adoption [11].

Gap 4: Limited Empirical Validation of Integrated Security Operational Processes. Existing literature has proposed numerous recommendations, conceptual architectures, and best practices for implementing DevSecOps. However, comprehensive empirical validation of integrated operational processes remains comparatively limited. Myrbakken and Colomo-Palacios highlighted that much of the current literature discusses DevSecOps from conceptual or practical perspectives without providing comprehensive evaluation of integrated operational frameworks [10]. More recently, Zhao et al. identified empirical validation as one of the research dimensions requiring further attention, particularly for studies involving broader operational integration and practical implementation [12]. Furthermore, Rajapakse et al. emphasized the need for additional empirical studies to evaluate integrated DevSecOps approaches in realistic operational environments [11].

1.5 Research objectives

Based on the research challenges and gaps identified in the previous sections, our objective is to design and develop a SysSecOps operational process for hybrid cloud environments that integrates system administration, continuous security, and software delivery into a unified workflow. The proposed process aims to improve the coordination between infrastructure management, security operations, and software development while supporting secure, automated, and efficient software deployment across heterogeneous cloud infrastructures.

To achieve this objective, the research is conducted with the following

specific objectives:

- To investigate the operational and security challenges associated with hybrid cloud environments and analyze the limitations of existing approaches in integrating system administration, software development, and continuous security.
- To design a SysSecOps operational process that incorporates continuous security throughout the software operational lifecycle by integrating infrastructure management, automated security assessment, policy enforcement, and deployment activities into a unified workflow.
- To develop a prototype SysSecOps system that implements the proposed operational process by integrating multiple security assessment tools, infrastructure automation, and continuous security mechanisms within a hybrid cloud environment.
- To establish a unified security assessment mechanism capable of aggregating security findings from heterogeneous security tools and supporting risk evaluation to facilitate security analysis and remediation prioritization.
- To evaluate the proposed SysSecOps system through representative hybrid cloud deployment scenarios and analyze its effectiveness in addressing the identified research challenges. The evaluation focuses on the feasibility of the proposed operational process, the integration of continuous security, the automation of security assessment, and the overall capability of the system to improve security operations within hybrid cloud environments.

1.6 Contributions

The main contributions of this research are summarized as follows:

- A SysSecOps operational process for hybrid cloud environments is proposed, integrating system administration, continuous security, and software delivery into a unified workflow. The proposed process addresses the operational challenges of heterogeneous infrastructure management, continuous security integration, and security automation identified in current hybrid cloud environments.

- A unified security assessment mechanism is developed by integrating multiple security analysis tools into a common security evaluation process. The proposed mechanism aggregates heterogeneous security findings and supports risk evaluation to improve security analysis and vulnerability prioritization. Its effectiveness is validated using a security dataset containing both vulnerable and secure software samples.

- A prototype SysSecOps system is designed and implemented to realize the proposed operational process within a hybrid cloud environment. The prototype integrates infrastructure automation, continuous security assessment, and automated deployment workflows. Furthermore, representative experimental scenarios are conducted to evaluate the feasibility and effectiveness of the proposed approach in improving continuous security integration and operational security management.

1.7 Thesis organization

The remainder of this thesis is organized as follows.

Chapter 2 reviews the literature related to this research. It presents the fundamental concepts of cloud computing and hybrid cloud environments, followed by an overview of DevOps, DevSecOps, and SysSecOps. Existing security assessment approaches, risk evaluation methods, and current research on secure software delivery are also reviewed to establish the

foundation and identify the limitations addressed by this research. Also, an AI-assisted code generation is introduced to increase code productivity.

Chapter 3 presents the proposed SysSecOps process for hybrid cloud environments. It describes the overall system architecture, the operational workflow, the unified security assessment mechanism, and the proposed risk evaluation process. This chapter also explains how continuous security is integrated throughout the software operational lifecycle.

Chapter 4 describes the implementation of the proposed SysSecOps system and evaluates its effectiveness. The implementation environment, prototype system, experimental scenarios, and evaluation methodology are presented, followed by the experimental results and performance analysis.

Chapter 5 discusses the findings obtained from the experimental evaluation. The proposed approach is analyzed with respect to the research objectives and existing studies, while its limitations and potential improvements are also discussed.

Finally, Chapter 6 concludes the thesis by summarizing the research, highlighting its main contributions, and suggesting directions for future work.

Chapter 2

Related work

2.1 Cloud computing

The most widely used definition of cloud computing was proposed by the National Institute of Standards and Technology (NIST). Mell and Grance define cloud computing as a model focusing on ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources that can be rapidly updated and released with minimal management effort or service provider interaction [4]. This definition has become the de facto standard adopted by both academia and industry.

According to NIST, cloud computing is characterized by five essential characteristics: on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service [4]. These characteristics help organizations dynamically allocating computing resources to workload demands while improving scalability and resource utilization. In addition, NIST defines cloud deployment into four models: private cloud, public cloud, community cloud, and hybrid cloud. Among these deployment models, hybrid cloud has gained many attentions because of its flexibility and scalability from public cloud services with the autonomy and security provided by private cloud infrastructures.

The widespread adoption of cloud computing has transformed modern software development and IT operations. Many organizations concentrate

on deploying native cloud applications using multiple automated development pipelines and distributed infrastructures. Thus, this makes security become an essential consideration throughout the software lifecycle. As a result, cloud computing serves as the technological foundation for the hybrid cloud environments and SysSecOps process.

2.2 Hybrid cloud

Hybrid cloud is one of the four cloud deployment models defined by the National Institute of Standards and Technology (NIST). According to Mell and Grance, a hybrid cloud is a combination of two or more distinct cloud infrastructures (private, community, or public) that still remain their unique characteristics but are joined together by standards, policies or proprietary technology [4]. Unlike public or private cloud deployments alone, hybrid cloud allows organizations to distribute workloads across multiple computing environments while maintaining interoperability between them. This deployment model helps organizations leveraging the scalability and elasticity of public cloud services while preserving sensitive applications and data within private cloud infrastructures to satisfy security, regulatory, or organizational requirements.

To maintain interoperability among heterogeneous cloud infrastructures, the National Institute of Standards and Technology further proposed the Cloud Computing Reference Architecture (CCRA), which describes many important actors, functional components, and interactions within cloud computing systems [5]. The architecture provides a standardized view of how cloud providers, cloud consumers, cloud brokers, cloud auditors, and cloud carriers interact to support cloud services. Within hybrid cloud environments, these components cooperate with each others to increase workload portability, resource orchestration, service management, and communication across multiple cloud platforms.

The flexibility offered by hybrid cloud has led to its widespread adop-

tion across both industry and academia. Recent industry reports indicate that hybrid cloud has become the deployment strategy for enterprise organizations, driven by the demand of balancing scalability, operational efficiency, regulatory compliance, and data governance [7], [8]. As cloud infrastructures continue to expand, organizations rely on hybrid cloud environments to support cloud-native applications, distributed services, and enterprise-scale software delivery.

Despite these advantages, hybrid cloud environments introduce significant operational complexity. Applications, services, and infrastructure components are distributed across heterogeneous platforms with different management interfaces, security policies, and deployment mechanisms. Maintaining consistent configurations, requiring uniform security policies, and coordinating operational activities across multiple cloud environments become more challenging as infrastructure complexity grows [5], [8]. These characteristics have motivated the adoption of operational methodologies that integrate infrastructure management, software delivery, and continuous security throughout the software operational lifecycle.

2.3 DevOps and DevSecOps

DevOps emerged as a software engineering methodology that promotes close collaboration between software development and IT operations through automation, continuous integration, continuous delivery, and continuous monitoring. Rather than treating development and operations as independent activities, DevOps integrates them into a continuous workflow that accelerates software delivery while improving consistent deployment and operational efficiency [13],[14].

The cloud computing paradigm has further accelerated the adoption of DevOps. Cloud platforms provide on-demand infrastructure, infrastructure-as-code (IaC), and automated deployment services that naturally is compatible with DevOps practices. These capabilities enable organizations to

provision infrastructure rapidly, automate software deployment, and monitor applications across distributed environments. Consequently, DevOps has become one of the predominant software delivery methodologies for cloud-native and hybrid cloud applications [13], [5].

Despite its advantages, DevOps primarily emphasizes development velocity, automation, and operational efficiency, while security activities have traditionally remained separate from the continuous development pipeline. Security assessments are frequently performed after software implementation or before deployment, increasing development costs and delaying software releases. As organizations deploy applications across cloud and hybrid cloud infrastructures, this separation becomes more problematic because security policies, infrastructure configurations, software dependencies, and cloud resources continuously evolve throughout the software operational lifecycle [11], [10].

To address these limitations, DevSecOps extends DevOps by integrating security practices into every stage of the software development and operational lifecycle. Rather than treating security as a separate verification phase, DevSecOps advocates continuous security through automated security testing, vulnerability assessment, compliance verification, infrastructure security, and continuous monitoring. The NIST Secure Software Development Framework (SSDF) similarly recommends incorporating security practices throughout software development to reduce vulnerabilities and improve software integrity [9]. Recent literature reviews further identify continuous security integration, automation, and security orchestration as the defining characteristics of DevSecOps while highlighting that these remain active areas of research, particularly for complex cloud environments [11], [12].

Although DevSecOps significantly strengthens software security, existing research primarily concentrates on integrating security into software development workflows. Comparatively less attention has been devoted to systematically coordinating software development, infrastructure manage-

ment, operational activities, and continuous security within heterogeneous hybrid cloud environments. This observation motivates the SysSecOps operational process proposed in this research, which extends existing DevSecOps practices by integrating system administration and infrastructure operations into a unified operational workflow for hybrid cloud environments.

2.4 Security Assessment and Risk Analysis

Security assessment plays a fundamental role in secure software development by identifying vulnerabilities throughout the software lifecycle before they can be exploited in production environments. Rather than treating security as a separate activity performed after software implementation, modern secure software development practices advocate continuous security assessment integrated into development, deployment, and operational processes. The NIST Secure Software Development Framework (SSDF) recommends incorporating automated security verification throughout the software development lifecycle to reduce software vulnerabilities and improve software integrity [9]. Similarly, the OWASP Software Assurance Maturity Model (SAMM) identifies continuous verification, security testing, and defect management as essential practices for establishing mature software security programs [15].

A wide range of security assessment techniques has been proposed to identify vulnerabilities from different perspectives. Static Application Security Testing (SAST) analyzes source code without execution to detect programming flaws, insecure coding patterns, and potential vulnerabilities during software development. In contrast, Software Composition Analysis (SCA) focuses on identifying vulnerabilities introduced through third-party libraries and software dependencies by comparing dependency information against publicly disclosed vulnerability databases. Additional techniques, including infrastructure and configuration analysis, evaluate deployment

environments and cloud infrastructure to identify security misconfigurations that may expose software systems to unnecessary risks. Because each technique targets different aspects of software security, they are generally considered complementary rather than interchangeable [9], [15].

As software systems become increasingly complex, organizations commonly employ multiple security assessment tools throughout the software lifecycle. Each tool produces findings using its own severity classification, confidence level, and reporting format, interpreting assessment results increasingly challenging. Consequently, modern vulnerability management extends beyond vulnerability detection to include vulnerability prioritization, enabling organizations to identify which security findings should be addressed first according to their potential impact and exploitation risk. NIST emphasizes that effective vulnerability management requires continuous identification, assessment, prioritization, remediation, and verification of security vulnerabilities throughout system operations rather than treating vulnerability scanning as an isolated activity [16].

Risk evaluation therefore has become an essential component of modern security assessment. The Common Vulnerability Scoring System (CVSS), maintained by the Forum of Incident Response and Security Teams (FIRST), provides a standardized approach for measuring the severity of software vulnerabilities based on exploitability, impact, and environmental factors [17]. Although CVSS has become the de facto standard for vulnerability severity assessment, it primarily evaluates individual vulnerabilities rather than supporting organizational remediation decisions. To address this limitation, the Stakeholder-Specific Vulnerability Categorization (SSVC) framework incorporates additional decision factors, including exploitation status, mission impact, and operational context, to assist organizations in prioritizing vulnerability remediation according to their specific environments [18].

Recent research has increasingly explored the application of machine learning techniques to vulnerability assessment and security risk predic-

tion. Unlike traditional rule-based scoring systems, machine learning models are capable of learning relationships among multiple security features and predicting vulnerability severity or exploitation likelihood from historical data. Various supervised learning algorithms have been investigated for this purpose, including Logistic Regression, Decision Trees, Random Forests, Support Vector Machines, and Gradient Boosting models. These approaches have been applied to vulnerability prioritization, exploit prediction, and software defect prediction, demonstrating that machine learning can improve the consistency and scalability of security risk evaluation by combining heterogeneous security indicators into a unified predictive model [19], [20], [21].

In practice, organizations often combine multiple security assessment techniques to obtain a more comprehensive understanding of software security. Static analysis, dependency analysis, infrastructure assessment, and compliance verification provide complementary perspectives on software risk, motivating the integration of heterogeneous security findings into unified security assessment workflows. Numerous tools have been developed to support these techniques, including Bandit for Python static analysis, Semgrep for pattern-based static analysis across multiple programming languages, and OWASP Dependency-Check for software dependency vulnerability analysis. While these tools improve vulnerability detection individually, effectively integrating their findings into a unified security evaluation process remains an important challenge for secure software operations. This challenge motivates the unified security assessment mechanism proposed in this research.

2.5 AI-Assisted Code Generation

Recent advances in generative artificial intelligence have significantly influenced modern software development practices. The introduction of the Transformer architecture established the foundation for large-scale

language modeling by replacing recurrent computation with an attention mechanism that captures long-range dependencies more effectively [22]. Building upon this architecture, large language models (LLMs) trained on massive text corpora demonstrated the ability to perform a wide range of language tasks with minimal task-specific supervision [23]. When trained on large collections of publicly available source code, these models further acquired the capability to generate syntactically correct and functionally relevant program code from natural language descriptions, enabling a new paradigm of AI-assisted software development [24].

Large language models trained on code have since been integrated into practical software development tools, allowing developers to generate code, complete functions, and translate natural language requirements into executable programs. These capabilities extend naturally to infrastructure-as-code (IaC), where cloud infrastructure is defined declaratively through configuration templates or programmatic frameworks. By generating IaC artifacts directly from natural language prompts, AI-assisted code generation can substantially accelerate infrastructure provisioning and reduce the manual effort required to author complex cloud configurations. This capability is particularly valuable in hybrid cloud environments, where infrastructure must be provisioned consistently across heterogeneous platforms with different deployment mechanisms.

Despite these advantages, AI-assisted code generation introduces important security concerns. Because large language models learn statistical patterns from large-scale training data that may contain insecure or outdated coding practices, the generated code does not inherently guarantee security, correctness, or compliance with organizational policies. Empirical studies have shown that a significant proportion of code produced by AI programming assistants contains security vulnerabilities. Pearce et al. evaluated code generated by GitHub Copilot and found that a substantial fraction of the generated programs exhibited security weaknesses corresponding to well-known vulnerability categories [25]. Similarly, controlled

user studies have observed that developers assisted by large language models tend to produce less secure code while simultaneously expressing greater confidence in its correctness, suggesting that AI assistance may inadvertently amplify security risks if left unchecked [26].

These concerns are particularly relevant in the context of infrastructure-as-code, where insecure configurations can directly expose cloud environments to attack. AI-generated IaC may introduce misconfigurations such as overly permissive access policies, unencrypted storage resources, or publicly exposed network endpoints, which can propagate throughout the infrastructure once deployed. In hybrid cloud environments, the impact of such misconfigurations is further amplified because infrastructure is distributed across multiple platforms with differing security controls. Consequently, the increased development velocity provided by AI-assisted code generation must be balanced with automated security verification to prevent insecure infrastructure from being deployed.

These observations motivate the integration of AI-assisted code generation with an automated security assessment mechanism within the operational workflow proposed in this research. Rather than treating AI-generated infrastructure code as trustworthy by default, the generated artifacts are systematically evaluated through security scanning and risk assessment before deployment. Furthermore, the security findings identified during evaluation can be provided back to the generation process as feedback, enabling iterative refinement of the generated code. By coupling AI-assisted code generation with continuous security verification, the proposed approach aims to leverage the productivity benefits of generative AI while mitigating the security risks associated with automatically generated infrastructure-as-code.

Chapter 3

Proposed System

3.1 System Architecture

This section presents the architecture of the proposed SysSecOps system for a hybrid cloud environment. The design places a quantitative, multi-scanner *security gate* between the synthesis and the deployment of Infrastructure-as-Code (IaC), so that no change is applied to either the public-cloud or the on-premises side until it has been statically analysed, costed, checked against live compliance state, and assigned a single risk score. The same engine governs both sides of the hybrid environment, which gives the process a uniform and reproducible security behaviour.

3.1.1 Design goals

The architecture is driven by the research objectives stated in Chapter 1 and refined into the following concrete design goals:

- **Gate before deploy.** Security evaluation must occur on the *synthesized* artifact (the CloudFormation template or the resolved Ansible play), not merely on the higher-level source, because the synthesized artifact is the last representation before any resource is provisioned.
- **One engine for the whole hybrid.** The AWS (public) side and

the on-premises (private) side must be scored with identical severity weights, deduplication rules, and decision thresholds, so that decisions are comparable across the trust boundary.

- **Graceful degradation.** A missing scanner binary or absent cloud credentials must never crash the pipeline; the affected component degrades to a well-defined "skipped" status while the gate still produces a decision.

- **Auditability.** Every decision (*pass*, *review*, *reject*) is persisted as an immutable record together with the acting identity, enabling post-hoc review.

- **Feedback-driven remediation.** When code is generated, gate findings are injected back into the generation prompt so that unsafe code is automatically regenerated rather than merely blocked.

- **Observability.** Every deployed target is continuously monitored and every gate decision is published as telemetry for dashboards and automated remediation.

3.1.2 Three-zone reference architecture

The system is organised into three cooperating zones, summarised in Table 3.1. Zone 1 produces IaC; Zone 2 evaluates and gates it; Zone 3 provisions the hybrid infrastructure and continuously observes it. The risk-scoring engine sits at the boundary of Zone 1 and Zone 2: it consumes scanner output and emits the decision that governs whether code advances to deployment or returns to Zone 1 for regeneration.

Figure 3.1 shows the three zones and the principal flow of control between them. The solid arrows denote the forward path of a change from prompt to deployed and monitored infrastructure; the dashed arrow denotes the remediation feedback that returns rejected generated code to

Table 3.1: The three zones of the proposed architecture.

Zone	Name	Responsibility
Zone 1	AI + IaC local development	Generate IaC from a natural-language prompt; local validation (`cdk synth`, `cdk diff`).
Zone 2	IaC security gate	Multi-scanner analysis, cross-source deduplication, risk scoring, and the deploy decision.
Zone 3	Hybrid infrastructure and operations monitoring	Provision AWS and on-premises resources; publish telemetry; run the drift/remediation operational loop.

Zone 1.

The three zones are intentionally loosely coupled. Zone 1 knows nothing about the internal scoring rules of Zone 2 — it only consumes the machine-readable findings that Zone 2 emits. Zone 3 knows nothing about how a decision was reached — it only consumes the decision event. This decoupling is what allows the generation backend, the set of scanners, and the monitoring stack to evolve independently, and it is the reason the same gate can serve both the public and the private side without modification. The remainder of this subsection describes each zone, and the tools it uses, in turn.

Zone 1 — AI-assisted generation and local validation

Zone 1 is where Infrastructure-as-Code originates. It exposes two entry surfaces — a command-line interface and a Streamlit operator console — and turns a natural-language prompt into synthesizable IaC. Code generation is delegated to a large-language-model backend behind a provider abstraction, so that the concrete model is a configuration choice rather than a code dependency; the reference implementation supports Amazon Bedrock and OpenRouter. The generated artifact is an AWS Cloud De-

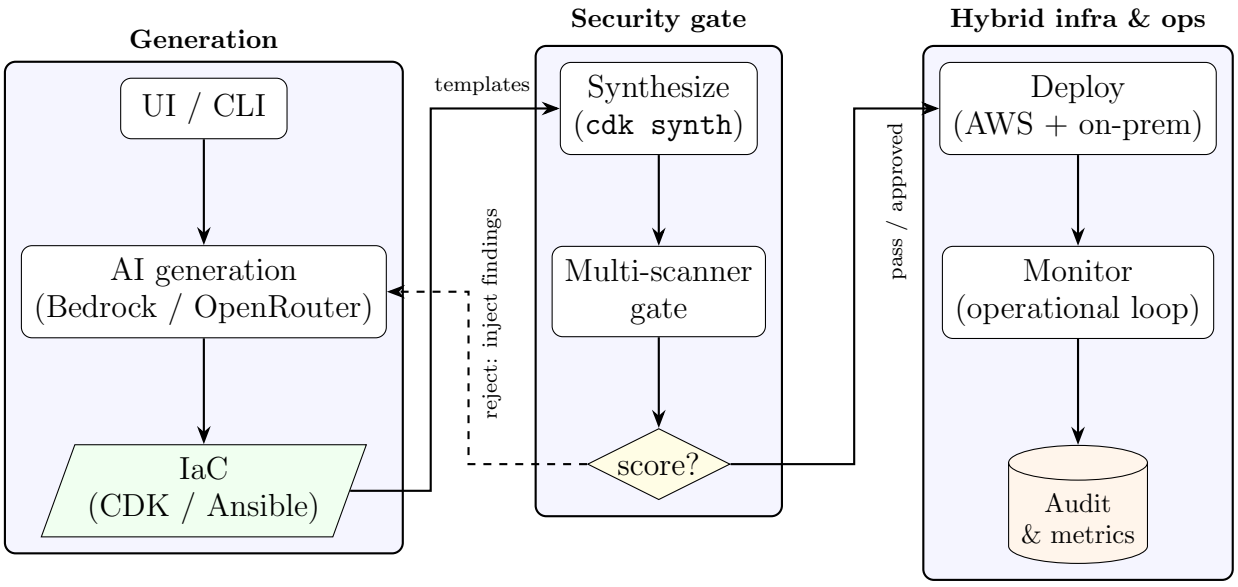


Figure 3.1: High-level three-zone architecture and end-to-end flow. Solid arrows are the forward path; the dashed arrow is the regeneration feedback loop.

velopment Kit (CDK) Python application for the public side and/or a set of Ansible playbooks for the private side.

Before any artifact leaves Zone 1, it is validated locally so that only well-formed input reaches the gate. For the public side, `cdk synth` synthesizes the CloudFormation template and `cdk diff` summarises the change against the currently deployed stack; for the private side, `ansible-playbook --syntax-check` (and an optional `--check` dry run) validates the plays. Zone 1 also closes the *generate* → *synth* → *gate* → *regenerate* feedback loop: when the gate rejects generated code, the findings are serialised back into the next prompt so that the model remediates its own output within a bounded attempt budget. Table 3.2 lists the tools used in this zone.

Zone 2 — the multi-scanner security gate

Zone 2 is the core contribution of the architecture: a quantitative security gate that consumes the synthesized artifact and emits a single de-

Table 3.2: Tools used in Zone 1 (generation and local validation).

Tool	**Role in Zone 1**
Amazon Bedrock / OpenRouter	Interchangeable LLM backends that generate CDK or Ansible code from a prompt.
AWS CDK (Python)	Expresses public-cloud infrastructure as code and synthesizes it to CloudFormation.
`cdk synth` / `cdk diff`	Produces the CloudFormation template and the change summary used for local validation.
Ansible	Expresses on-premises configuration as playbooks.
`ansible-playbook --syntax-check`	Validates playbook syntax (and, optionally, a dry-run `--check`) before gating.

cision. It runs a set of independent, best-of-breed scanners, normalises their heterogeneous outputs into a common finding schema, deduplicates findings that several tools report for the same resource/template/category, aggregates the surviving findings into a numeric risk score, and maps that score to *pass*, *review*, or *reject*. Each scanner is wrapped in an adapter that never raises: a missing binary or an absent credential degrades to a well-defined *skipped* status, and the gate still produces a decision (the graceful-degradation contract).

The tools are deliberately complementary and cover four dimensions of risk — *misconfiguration*, *cost*, *live compliance*, and *code-level insecurity*. Static policy-as-code analysis (Checkov) and CloudFormation validation (`cfn-lint`) inspect the public-side template; `ansible-lint` inspects the private-side plays; a built-in regular-expression scanner detects hard-coded secrets; Infracost supplies the estimated monthly cost that drives the cost dimension; AWS Config supplies the count of live non-compliant rules; and a machine-learning code-risk model (trained on Bandit and Semgrep features, see Section 3.3) contributes a probability of insecurity. Table 3.3 lists these tools and the signal each contributes.

Table 3.3: Tools used in Zone 2 (the security gate) and the signal each contributes.

Tool	**Signal**	**Dimension**
AWS CDK (`cdk synth`)	Emits the CloudFormation template that is the analysed artifact.	pre-validation
`cfn-lint`	CloudFormation validity and best-practice findings.	misconfig.
Checkov	Policy-as-code misconfiguration findings on the templates.	misconfig.
`ansible-lint`	Best-practice and security findings on the on-premises plays.	misconfig.
Regex secret scanner (built-in)	Hard-coded credentials and secrets.	misconfig.
Infracost	Estimated monthly cost / cost delta of the change.	cost
AWS Config	Count of non-compliant live-compliance rules.	compliance
ML code-risk model (Bandit + Semgrep → logistic regression)	Probability that the generated code is insecure, converted to points.	code risk

Zone 3 — hybrid infrastructure and operations

Zone 3 provisions the hybrid infrastructure that survived the gate and continuously observes it. The public side is provisioned on Amazon Web Services (AWS) by deploying the CDK-synthesized CloudFormation; the private side consists of on-premises Linux nodes configured by Ansible. After a successful deployment, a three-layer monitoring mesh is attached to the public resources and the on-premises nodes are registered so that both sides are watched by a single operational loop.

On the public side, AWS Systems Manager (SSM) provides Run Command, State Manager, Patch Manager, Inventory, and Parameter Store; Amazon EventBridge carries compliance and stack-rollback events; Amazon CloudWatch stores the gate metrics, alarms, dashboards, and Application Insights views; AWS CloudTrail records the API audit trail; an AWS Lambda function implements the event-driven operational loop (drift reaction and pipeline re-trigger); and Amazon SNS delivers operator notifications. On the private side, the SSM Agent together with an SSM *hybrid activation* registers each on-premises node as a managed instance (`mi-*`), so the same Run Command, drift-detection, patch, and compliance machinery used for EC2 also covers the on-premises fleet without a separate agent stack. Table 3.4 summarises the tooling.

3.1.3 End-to-end data flow

A single request flows through the system as an evolving chain of artifacts:

1. A natural-language prompt (or an existing, "bring-your-own" project) enters the system through the command-line interface or the operator console.

2. Zone 1 produces IaC: a synthesizable AWS Cloud Development Kit (CDK) Python application and/or a set of Ansible playbooks.

3. The IaC is synthesized: `cdk synth` emits CloudFormation templates, while `ansible-playbook --syntax-check` validates the playbooks.

4. Zone 2 evaluates the synthesized artifact with several independent scanners, normalises their outputs into a common finding schema, deduplicates across sources, and computes a risk score.

5. The decision function maps the score to *pass*, *review*, or *reject*. The

Table 3.4: Tools used in Zone 3 (hybrid infrastructure and operations).

Side	**Tool / service**	**Role**
Public	AWS CDK / CloudFormation	Provisions and updates the public-cloud resources.
Public	EC2, S3, RDS, VPC	The workload substrate that the CDK app declares.
Public	AWS Systems Manager	Run Command, State Manager, Patch, Inventory, and Parameter Store (the gate-decision store).
Public	Amazon EventBridge	Event bus for compliance-change and stack-rollback events.
Public	Amazon CloudWatch	Gate metrics, alarms, dashboards, and Application Insights.
Public	AWS CloudTrail	Multi-region API audit trail feeding CloudWatch Logs.
Public	AWS Lambda	The ops-loop function: drift reaction and pipeline re-trigger.
Public	Amazon SNS	Operator notifications for alarms and rollbacks.
Private	Ansible	Configuration management of the on-premises Linux nodes.
Private	SSM Agent + hybrid activation	Registers each node as an `mi-*` managed instance so it joins the same operational loop.
Shared	Secure connectivity link	Joins the two sides through one of several options (see Section 3.1.4).

full gate report is persisted, and an approval or rejection record is written.

6. On *pass* (or an approved *review*), the change is deployed; on *reject*, generated code is optionally regenerated with the findings injected back into the prompt.

7. Zone 3 records the outcome, publishes metrics and events, and attaches post-deployment security monitoring.

Figure 3.2 traces the same request as a chain of artifacts, making explicit how each stage transforms its input into a more concrete representation. The important property is that the security decision is taken on the *synthesized template*, not on the prompt or the high-level source: this is the last artifact before provisioning and therefore the earliest point at which the exact resources, their properties, and their relationships are fully known.

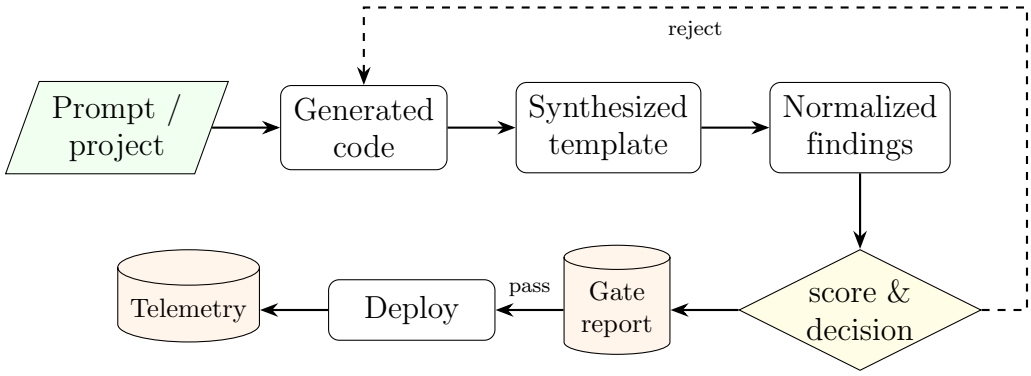


Figure 3.2: Artifact lifecycle of a single request. The decision is computed on the synthesized template; a rejection feeds findings back into generation.

3.1.4 Hybrid deployment topology

The public side is provisioned on Amazon Web Services (AWS) through CDK-synthesized CloudFormation. The private side consists of on-premises Linux nodes managed by Ansible. The two sides are joined by an encrypted link, and the architecture deliberately treats *how* that link is established as a pluggable concern: Section 3.1.4 surveys the available connectivity options. The on-premises nodes are registered as AWS Systems Manager (SSM) managed instances so that their compliance state is visible to the same operational loop that watches the cloud side. No public inbound administrative access is required: the control plane traverses the encrypted link, which keeps the hybrid trust boundary small and observable. Figure 3.3 depicts the topology with the connectivity method shown as an abstract secure channel.

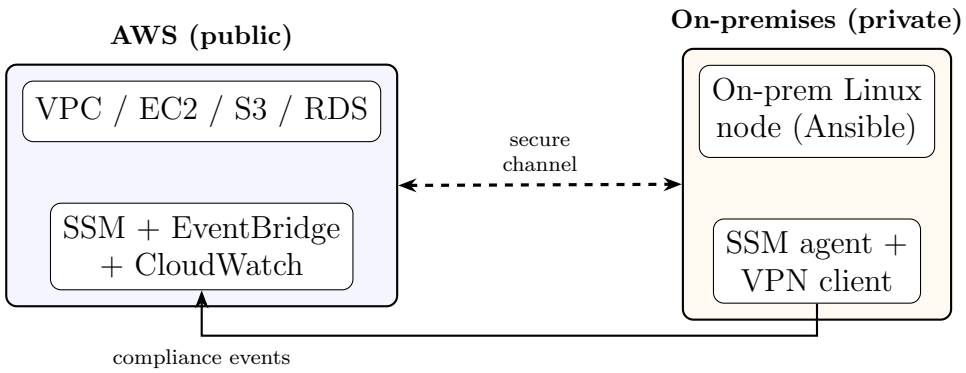


Figure 3.3: Hybrid deployment topology: AWS resources and on-premises nodes joined by a secure channel (any of the options in Table 3.5), with on-premises nodes registered as SSM managed instances.

Connectivity options between the public and private sides

Joining the public and private sides is a design choice with several viable options rather than a single fixed mechanism. The architecture treats connectivity as a pluggable concern: the gate, the SSM registration, and the operational loop are indifferent to *how* the link is established, as long as the on-premises nodes can reach the regional AWS endpoints and the two sides can exchange traffic securely. Table 3.5 compares the practical options, ordered from the lightest-weight software overlay to a dedicated physical link, so that a deployment can select the method that matches its latency, bandwidth, cost, and compliance requirements.

These options are not mutually exclusive: a lightweight mesh overlay can carry workload and control traffic while an SSM hybrid activation provides the operational loop with management-plane reach, and the heavier options (IPsec Site-to-Site VPN, Direct Connect, or a Transit Gateway hub) can be substituted where standards-based encryption, dedicated bandwidth, or multi-site routing is required. Crucially, the choice of connectivity is orthogonal to the rest of the pipeline: none of these methods changes the gate, the risk scoring, or the deployment-governance behaviour, which is what allows the architecture to remain portable across different hybrid networking substrates.

Table 3.5: Options for connecting the public (AWS) and private (on-premises) sides.

Method	**Mechanism**	**Trade-offs**
Mesh overlay VPN (Tailscale / WireGuard)	Userspace WireGuard tunnels with automatic NAT traversal and identity-based access control lists; each node runs a lightweight agent.	Fast to set up, no public inbound ports, per-node identity; depends on a software agent and a coordination service.
AWS Site-to-Site VPN (IPsec)	Managed IPsec tunnels between an on-premises customer gateway device and an AWS virtual private gateway or transit gateway.	Standards-based and fully encrypted over the public Internet; needs a compatible gateway device and is bandwidth-limited per tunnel.
AWS Direct Connect	A dedicated private physical circuit from the on-premises data centre into AWS, bypassing the public Internet.	Consistent low latency and high, stable bandwidth; higher cost and provisioning lead time, and usually paired with a VPN for encryption.
AWS Transit Gateway / VPN CloudHub	A managed routing hub that interconnects many VPCs and multiple on-premises sites in a hub-and-spoke topology.	Centralises and scales routing across sites; adds a managed routing tier and its associated cost.
SSM hybrid activations over public endpoints	No network tunnel: the SSM Agent reaches the regional Systems Manager endpoints directly over TLS 443.	Simplest option for management-plane reach only; provides no private data-plane connectivity between workloads.

3.1.5 Component map and interfaces

Table 3.6 lists the top-level software modules and their responsibilities. The modules communicate through stable contracts rather than shared

internal state, which keeps the generation, evaluation, orchestration, and monitoring concerns decoupled.

Table 3.6: Top-level modules and their responsibilities.

Module	**Responsibility**
`generation/`	Zone 1 generation: provider-abstracted code generation (Bedrock / OpenRouter) and the CDK regeneration loop.
`security_gate/`	Zone 2 gate: template and playbook analysis, scanner adapters, deduplication, risk scoring, and report assembly.
`execution/`	Subprocess execution backbone with a stable result contract.
`pipeline/`	Orchestration and governance: synth–gate–diff–deploy, the approval audit trail, credentials, and the operational-loop emitters.
`monitoring/`	CloudTrail, CloudWatch metrics and dashboards, the operational-loop Lambda, and on-premises SSM registration.
`scripts/`	Command-line entry points for the CDK-only and the hybrid flows.
`ui/`	Streamlit operator console and the multi-project registry.
`risk_scoring/`	Training and evaluation code for the machine-learning code-risk model consumed by the gate.

3.1.6 Key data contracts

Three contracts hold the system together and are deliberately kept stable so that independent consumers (the CLI, the console, the dashboards) do not break:

- **Gate report.** A JSON document containing the run identifier,

timestamp, numeric `score`, `decision`, human-readable `message`, a per-component `components` breakdown, the deduplicated `findings`, and a `scanner_status` map that records whether each scanner ran, was skipped, or was unavailable.

- **Execution result.** Every external command returns exactly `{return_code, output}`, with standard output and standard error merged to preserve event ordering for the console and the logs.
- **Gate-decision event.** A compact event (target name, score, decision, timestamp) is persisted to SSM Parameter Store and published to Amazon EventBridge, keyed by the target name (a stack name for CDK, or `ansible-<node>` for the on-premises side) so that the two sides never collide.

3.1.7 Security and trust model

Credentials are never passed as command-line arguments; they are resolved through the standard provider chains and injected into subprocesses through the environment. AWS access follows the default credential chain with a single-sign-on fallback, and generation-provider keys are read from the environment. Deployed workloads follow the principle of least privilege, and each privileged action is recorded in an append-only audit trail. The uniform graceful-degradation contract guarantees that the absence of a tool or a credential weakens visibility but never silently disables the gate.

3.2 Proposed Methodology

3.2.1 Approach overview

Each hybrid-cloud challenge identified in Chapter 1 is mapped to a concrete module that addresses it. Table 3.7 makes this mapping explicit

and provides the traceability from problem to solution that motivates the remainder of the section.

Table 3.7: Mapping from hybrid-cloud challenges to solution modules.

Challenge	**Module / mechanism**
Weak security linkage between public and private cloud	One gate and one risk engine applied to both the CDK and the Ansible side.
Absence of a clear, repeatable process	The synth–gate–decide–deploy pipeline with fixed thresholds and audit records.
Fragmented tooling with inconsistent output	Scanner adapters that normalise findings into a common schema, followed by cross-source deduplication.
Misconfiguration risk amplified by fast AI generation	The generate–gate–regenerate feedback loop that injects findings back into the prompt.
Context-free prioritisation	A risk score that combines severity with cost, live compliance, and a machine-learning code-risk signal.

3.2.2 AI-assisted generation and the regeneration loop

Zone 1 generates synthesizable CDK Python code from a natural-language prompt. Code generation is placed behind a provider abstraction so that two backends — a managed foundation-model service and a hosted inference API — are interchangeable behind a single provider flag and share an identical generation contract. Model defaults are centralised and can be overridden explicitly.

Generation alone does not guarantee secure output, so it is wrapped in a closed loop. The loop generates code, synthesizes it, gates it, and — if the

gate rejects — rebuilds the prompt with the machine-readable findings injected before retrying, up to a bounded number of attempts. Each attempt is persisted for reproducibility under `logs/cdk_regen/<run_id>/attempt_<N>/` (the prompt snapshot, the generated code, the synthesis output, and the gate report), and the successful attempt is promoted to a `passed/` folder. Algorithm 1 formalises the loop.

Algorithm 1 Generate–synthesize–gate–regenerate loop

Require: prompt q, maximum attempts N
Ensure: gated CDK application, or failure after N attempts

$findings \leftarrow \emptyset$
for $i \leftarrow 1$ to N **do**
  $code \leftarrow \textsc{Generate}(q, findings)$
  write *code* to the CDK application
  $t \leftarrow \textsc{Synth}(code)$ ▷ emit CloudFormation templates
  if SYNTH failed **then**
    $findings \leftarrow$ synthesis error
    continue
  $report \leftarrow \textsc{Gate}(t)$
  if $report.decision \neq reject$ **then**
    return *report* ▷ pass or review
  $findings \leftarrow report.findings$ ▷ injected into the next prompt
return failure

Figure 3.4 presents the same loop as a flowchart, emphasising the two exit conditions: an accepting decision (*pass* or *review*) leaves the loop with a gated application, whereas exhausting the attempt budget N leaves it with a failure that surfaces to the operator.

3.2.3 Pre-validation

Before any security analysis, each side is validated syntactically. On the CDK side, `cdk synth` must succeed (otherwise the loop treats the failure as feedback), and `cdk diff` previews the change. On the Ansible side, `ansible-playbook --syntax-check` validates the plays and

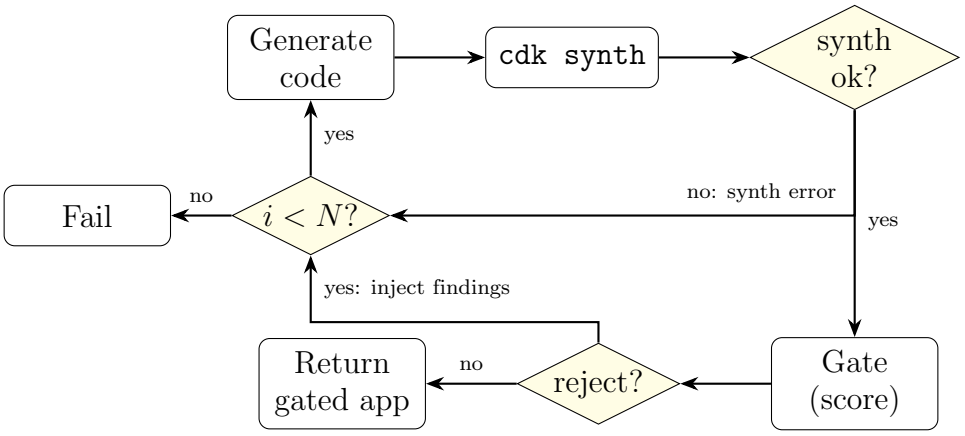


Figure 3.4: Regeneration loop as a flowchart. The loop exits on an accepting decision or when the attempt budget N is exhausted.

`ansible-playbook --check --diff` performs a dry run. This staging (*validate* → *dry run* → *deploy*) is symmetric across the two sides and ensures the gate only ever scores artifacts that are at least well formed.

3.2.4 Security evaluation and cross-source deduplication

Zone 2 analyses the synthesized artifact with several independent scanners, each wrapped in an adapter that normalises its output into a common finding schema with the fields `severity`, `source`, `message`, `resource_id`, `template`, and `category`. Because several tools frequently report the same underlying issue (for example, an open administrative port may be flagged by both a heuristic check and a policy scanner), the findings are deduplicated by the key (`resource_id`, `template`, `category`). When duplicates collide, the highest severity is retained and the source labels are merged. Deduplication is essential: without it, redundant reports would inflate the score and distort the decision. Figure 3.5 shows the analysis pipeline: the synthesized artifact fans out to the parallel scanner adapters, whose normalized findings are merged, deduplicated, and passed to the risk-scoring engine.

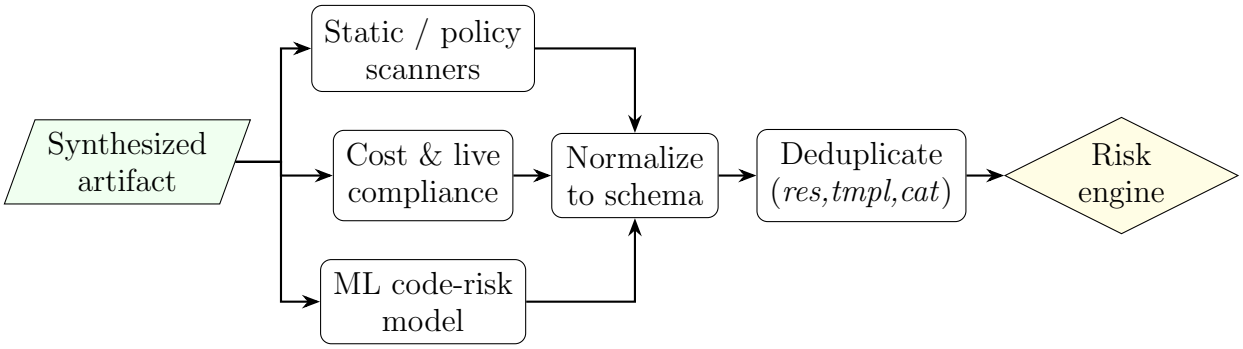


Figure 3.5: Security-evaluation pipeline: parallel scanners, normalization to a common schema, cross-source deduplication, and aggregation by the risk-scoring engine.

3.2.5 Risk-scoring engine (overview)

The deduplicated findings, together with the cost, live-compliance, and machine-learning code-risk signals, are aggregated by the gate into a single numeric score that is then mapped to one of three decisions — *pass*, *review*, or *reject*. Because this aggregation is the analytical heart of the proposed process, it is presented in full in Section 3.3, which develops the multi-factor formulation, the additive scoring implementation, the machine-learning code-risk model, the deduplication-driven explainability, and a worked example. Here it suffices to note that the score is an *additive* combination of weighted findings, banded cost, live compliance violations, and a bounded machine-learning contribution, and that the decision thresholds are fixed so that identical inputs always yield identical decisions on both sides of the hybrid environment.

3.2.6 Deployment governance

The decision governs deployment. A *pass* deploys automatically. A *review* requires an explicit approval, which is recorded together with the approver's cloud identity, the run identifier, the score, and the decision. A *reject* never deploys; when the code was generated, the regeneration loop

is invoked, and otherwise a rejection record is written for the author to act on. Approval and rejection records are append-only, which yields the immutable audit trail required by the design goals.

3.2.7 Monitoring and the operational loop

After a successful deployment, Zone 3 attaches post-deployment monitoring and closes an operational loop. Each gate decision is persisted to SSM Parameter Store, published as an EventBridge event, and emitted as CloudWatch metrics and structured logs. An event-driven function reacts to compliance drift and rollback events, enabling automated remediation. Because the on-premises nodes are registered as managed instances, their compliance events flow through the same loop, so the entire hybrid environment is observed uniformly rather than as two disconnected halves.

3.3 Risk-Scoring Engine

3.3.1 Motivation and limitations of severity-only scoring

Industry practice prioritises vulnerabilities almost exclusively by technical severity, typically a Common Vulnerability Scoring System (CVSS) base score. For infrastructure changes in a hybrid cloud this is insufficient for three reasons:

- **Severity ignores blast radius and cost.** An open security-group rule on an internet-facing load balancer and the same rule on an isolated subnet may share a severity label yet differ enormously in exposure. Likewise, a change that quietly provisions expensive resources carries an operational risk that no security-only score captures.

- **Single tools have blind spots.** A policy scanner, a template linter, a cost estimator, a live-compliance service, and a code-level model each see a different slice of the same change. Relying on any one of them yields systematic false negatives.
- **AI generation shifts the risk profile.** When code is produced quickly by a language model, the dominant failure mode is not a novel exploit but a plausible misconfiguration. A signal that estimates the *probability* that generated code is insecure is therefore as valuable as any individual rule.

The engine addresses these limitations by combining several orthogonal signals into one additive score with a fixed, auditable decision boundary, so that prioritisation reflects severity and exposure, cost, live compliance, and a learned code-risk estimate.

3.3.2 Multi-factor formulation

Conceptually, the risk of a change is a weighted combination of five factors:

- **Severity** (S): the technical seriousness of each finding.
- **Exposure** (X): how reachable the affected resource is (for example, public versus private).
- **Exploitability** (E): how readily a finding can be abused.
- **Confidence** (C): how certain the detecting tool is, which discounts low-confidence heuristics.
- **Cost** (K): the financial blast radius of the change.

A general multi-factor risk of a change with finding set F can be written as

$$R \;=\; \sum_{f \in F} \alpha\, S_f \cdot X_f \cdot E_f \cdot C_f \;+\; \beta\, K, \tag{3.1}$$

where α and β balance the finding-driven term against the cost term. This formulation is the design intent; the implemented engine (next subsection) is a deliberately simplified, *additive* realisation of it that keeps the score transparent and reproducible while folding exposure, exploitability, and confidence into the calibrated severity weights and the live-compliance term.

3.3.3 Implemented additive scoring

The gate implements Equation 3.1 as an additive sum of four components that are easy to implement and cheap to compute. Each deduplicated finding contributes points according to its severity (Table 3.8); a monthly cost increase contributes points in bands; each non-compliant live rule contributes a fixed number of points; and the machine-learning model contributes points that rise with its predicted probability of insecurity.

Table 3.8: Severity-to-points mapping used by the gate.

Severity	**Points**
Critical	20
High	10
Medium	5
Low	1

Formally, let F be the set of deduplicated findings, $\mathrm{sev}(f)$ the severity of finding f, and $w(\cdot)$ the mapping in Table 3.8. Let Δ be the estimated monthly cost increase in US dollars, $c(\Delta)$ the banded cost contribution, v the number of non-compliant live-compliance rules, and $p \in [0, 1]$ the predicted probability of insecurity from the machine-learning model. The total score S is

$$S \;=\; \underbrace{\sum_{f \in F} w\big(\mathrm{sev}(f)\big)}_{\text{severity}} \;+\; \underbrace{c(\Delta)}_{\text{cost}} \;+\; \underbrace{5\,v}_{\text{compliance}} \;+\; \underbrace{\mathrm{round}\big(p^{\gamma} \cdot P_{\mathrm{ml}}\big)}_{\text{ML code-risk}}, \tag{3.2}$$

where $P_{\mathrm{ml}} = 85$ is the maximum contribution of the machine-learning component, attained at $p = 1$, and $\gamma = 2.5$ governs how that contribution accrues below it.

The decision function maps S to one of three outcomes using the thresholds in Table 3.9:

$$\mathrm{decision}(S) = \begin{cases} \textit{pass} & \text{if } S \leq 20, \\ \textit{review} & \text{if } 20 < S \leq 80, \\ \textit{reject} & \text{if } S > 80. \end{cases} \tag{3.3}$$

Table 3.9: Decision thresholds and their operational meaning.

Decision	**Score band**	**Action**
Pass	≤ 20	Auto-deploy; no human review required.
Review	21–80	Manual approval required before deployment.
Reject	> 80	Do not deploy; regenerate (if generating) or return to the author.

Figure 3.6 summarises the engine: the four component sub-scores are summed and compared against the two thresholds to select the decision.

3.3.4 Learned multi-source risk module

The learned risk module supplements deterministic assessment rules with a statistical estimate of whether an infrastructure-related code artifact

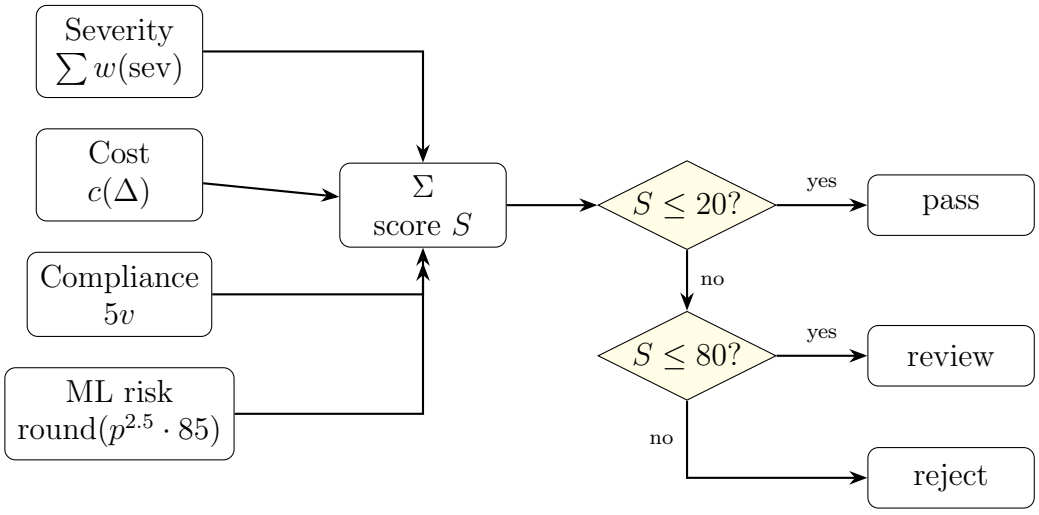


Figure 3.6: The additive risk-scoring engine: four orthogonal sub-scores are summed and the total is compared against two fixed thresholds to yield the decision.

resembles previously observed insecure examples. Its purpose is not to replace the individual assessment components, but to combine their signals into an additional probability $p \in [0, 1]$ for use in Equation 3.2. This design allows several assessment sources to contribute to a common risk estimate even when their native outputs use different formats, taxonomies, and confidence scales.

The module consists of four general stages: construction of a labelled corpus, normalisation of assessment outputs, training of a probabilistic classifier, and integration of the predicted probability into the unified score. The concrete data sources, assessment tools, model type, and training parameters are implementation choices and are therefore described in Chapter 4.

Labelled corpus

The training corpus contains artifacts representing two classes: examples with known security weaknesses and reference examples without known weaknesses. Each artifact is assigned a binary label, where label 1 represents the insecure class and label 0 represents the reference class. Fil-

tering rules are applied consistently to reduce irrelevant variation caused by generated files, test artifacts, empty inputs, or differences in artifact size.

The corpus must reflect the detection scope of the assessment sources used to construct the features. If a weakness cannot produce any observable signal under the selected assessment methods, the classifier cannot learn a reliable relationship for that weakness. Consequently, the output is interpreted as the probability of visible insecurity, rather than as proof that an artifact is secure or insecure in every semantic sense.

Common feature representation

Every assessment source is connected through an adapter that converts its native output into the common finding schema. For each artifact, the normalised findings are then reduced to a fixed-length feature vector. The vector may contain counts grouped by source, severity, confidence, category, and total finding volume. Using aggregated numeric features avoids dependence on tool-specific message text and allows heterogeneous sources to be processed by the same model.

Let m denote the number of assessment sources and let G denote the set of common finding groups. For artifact k, a general feature vector can be expressed as

$$\mathbf{x}_k = \left[n_{k,1,g_1}, \ldots, n_{k,1,g_{|G|}}, \ldots, n_{k,m,g_1}, \ldots, n_{k,m,g_{|G|}} \right]^\top, \tag{3.4}$$

where $n_{k,j,g}$ is the number of findings reported for artifact k by source j in group g. The grouping definition is fixed before training and reused unchanged during inference. This common representation is the mechanism that enables the module to combine signals from multiple assessment sources without binding the methodology to particular products.

Training procedure

The labelled corpus is divided into disjoint training and evaluation partitions while preserving the class distribution. Any preprocessing transformation, such as feature scaling, is fitted using only the training partition and then applied unchanged to the evaluation and operational inputs. If the class distribution is imbalanced, weighting or sampling strategies may be used to prevent the majority class from dominating the model.

A probabilistic binary classifier is fitted to estimate $P(y_k = 1 \mid \mathbf{x}_k)$, where y_k is the label of artifact k. The trained model and every preprocessing artifact are persisted together. This preserves *training–inference parity*: the operational module uses the same feature grouping, ordering, transformations, and model parameters that were used during training.

Inference and score integration

During inference, the module constructs a feature vector for each artifact and obtains a probability from the trained classifier. For a change containing several artifacts, the module uses the maximum probability:

$$p = \max_{k \in \mathcal{A}} \mathcal{M}(\mathbf{x}_k), \tag{3.5}$$

where $\mathcal{A}$ is the set of evaluated artifacts and $\mathcal{M}$ is the trained probability model. The maximum operator prevents a high-risk artifact from being masked by a larger number of low-risk artifacts. The probability is converted into the bounded machine-learning contribution in Equation 3.2, while the per-artifact probabilities remain available in the gate report for explanation and remediation.

Unavailable assessment sources are recorded explicitly rather than silently treated as successful checks. The module may continue with the remaining sources when permitted by policy, but the missing evidence must be visible in the report and may require additional review.

3.3.5 Deduplication and explainability

Two properties make the score trustworthy. First, *deduplication* guarantees that no single underlying issue is counted twice: findings are keyed by (`resource_id`, `template`, `category`), and colliding findings are merged, keeping the highest severity and unioning the source labels. Without this step, an issue reported by three tools would triple its severity contribution and could push an otherwise acceptable change over a threshold.

Second, *explainability* follows directly from the additive form of Equation 3.2: because the score is a sum of clearly labelled components, the gate report includes a per-component breakdown that attributes exactly how many points came from severity, cost, compliance, and the machine-learning model. An operator reviewing a *review* or *reject* decision can therefore see *why* the score landed where it did and which component to remediate, rather than being handed an opaque number.

3.4 Prototype Implementation

3.4.1 Technology stack and repository layout

The public side is defined with the AWS Cloud Development Kit for Python and synthesized to CloudFormation; the private side is defined with Ansible playbooks. Static analysis relies on established open-source scanners, cost estimation on a cost-analysis tool, and live compliance on the cloud provider's configuration service. The operator console is built with Streamlit. Table 3.10 lists the directories that implement each concern.

Figure 3.7 shows how these directories depend on one another at runtime. The command-line entry points in `scripts/` sit at the top; they call into `pipeline/` for orchestration, which in turn drives generation (`generation/`), evaluation (`security_gate/`), and monitoring (`monitoring/`). Every directory that runs an external process funnels through the single

Table 3.10: Repository layout and the concern each directory implements.

Directory	Contents
`generation/`	Provider backends and the CDK regeneration loop (`run_cdk_regen.py`).
`security_gate/`	The gate (`iac_security_gate.py`) and the scanner adapters under `security_gate/scanners/`.
`execution/`	The subprocess execution helper (`exec_code.py`).
`pipeline/`	Orchestration, credentials, and operational-loop emitters.
`monitoring/`	CloudTrail, dashboards, the operational-loop function, and SSM registration.
`scripts/`	`run_cdk_pipeline.py` and `run_hybrid_pipeline.py`.
`ui/`	The Streamlit console and the multi-project registry.
`risk_scoring/`	Dataset preparation and model training.

execution helper in `execution/`, and the operator console in `ui/` reuses the same entry points rather than duplicating logic.

3.4.2 Generation layer

Two interchangeable backends implement the generation contract behind a common interface, so that the choice of provider is a configuration flag rather than a code change. Both expose the same functions to call the model, extract the code from the response, and save the result, and both centralise their default model identifiers while allowing an explicit override. Provider-specific error handling (for example, quota and throttling detection) is isolated behind the provider boundary, and credentials are read from explicit arguments first and then from the environment. The regeneration loop described in Algorithm 1 orchestrates these backends and

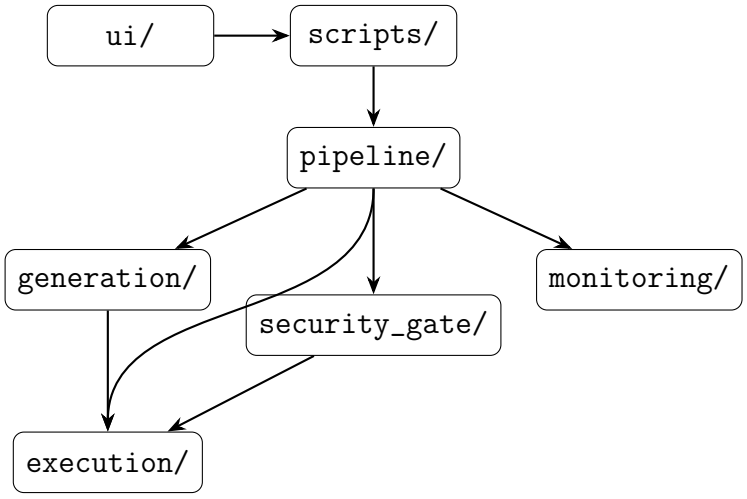


Figure 3.7: Runtime module dependencies. Entry points drive orchestration, which drives generation, evaluation, and monitoring; all external processes pass through the single execution helper.

persists the per-attempt artifacts.

3.4.3 Security gate and scanner adapters

The gate is implemented in `iac_security_gate.py`, which collects the synthesized templates, runs the heuristic checks and the scanner adapters, deduplicates the findings, computes the score of Equation 3.2, applies the decision function of Equation 3.3, and writes the gate report. Each scanner lives in its own adapter and returns a dictionary with at least a status and a message; no adapter ever raises, so a missing tool degrades to a well-defined status while the gate proceeds. Table 3.11 lists the adapters.

An abbreviated gate report is shown in Listing 3.1. The `components` object exposes the per-dimension contribution to the score, which makes the decision explainable, and the `scanner_status` object records which tools ran.

```
{
  "run_id": "cdk_20260614T093934Z",
  "timestamp": "2026-06-14T09:39:42+00:00",
  "score": 50,
  "decision": "review",
```

Table 3.11: Scanner adapters and the signal each contributes.

Adapter	Signal
`checkov_adapter.py`	Policy-as-code findings on the synthesized templates.
`cfn_lint_adapter.py`	CloudFormation best-practice and validity findings.
`infracost_adapter.py`	Estimated monthly cost, used as the cost dimension.
`aws_config_adapter.py`	Count of non-compliant live-compliance rules.
`ansible_lint_adapter.py`	Findings on the on-premises playbooks.
`secret_scan_adapter.py`	Regular-expression secret detection (no external tool).
`ml_risk_adapter.py`	Logistic-regression probability of insecurity, converted to points.

```
  "message": "Manual review required before deployment.",
  "components": {"severity": 30, "cost": 10, "aws_config": 10, "ml_risk":
   0},
  "findings": [
    {"severity": "high", "source": "checkov", "category": "sg_ssh_open",
     "resource_id": "MySG", "template": "App.template.json",
     "message": "Security group allows ingress on port 22 from 0.0.0.0/0"}
  ],
  "scanner_status": {"checkov": "ok", "cfn_lint": "ok",
                     "infracost": "ok", "aws_config": "ok", "ml_risk": "ok"
   }
}
```

Listing 3.1: Abbreviated gate report.

3.4.4 Orchestration and governance

Two command-line entry points drive the pipeline. The CDK-only entry point, `run_cdk_pipeline.py`, orchestrates bootstrap, synthesis, gating, diff, and deployment for the public side, and can optionally invoke the regeneration loop on a reject. The hybrid entry point, `run_hybrid_pipeline.py`,

runs the same gate, risk-scoring, deployment, and observability workflow over both a bring-your-own CDK directory and a bring-your-own Ansible directory — with no code-generation step — gating each side independently with the same engine and writing a combined report. The orchestration routes all external commands through the execution backbone and honours the deployment-governance rules of Section 3.2.6. Listing 3.2 shows representative invocations.

```
# CDK-only: synth, gate, and deploy if the gate allows
python scripts/run_cdk_pipeline.py --project-dir generated_cdk --deploy

# Hybrid: gate both sides; scope the Ansible scan to files changed since
    HEAD~1
python scripts/run_hybrid_pipeline.py \
    --cdk-path examples/hybrid-demo/cdk \
    --ansible-path examples/hybrid-demo/ansible \
    --base-ref HEAD~1

# Read-only status of every gated target
python scripts/run_hybrid_pipeline.py --query-status
```

Listing 3.2: Representative pipeline invocations.

On the hybrid side, the Ansible scan can be scoped to the files changed since a given git reference, which limits analysis to the modified plays. The decision returned by each entry point is surfaced as a distinct process return code, so that both the console and any external continuous-integration system can react to *pass*, *review*, and *reject* without parsing free text.

Figure 3.8 traces the hybrid pipeline as a sequence of interactions between the orchestrator, the gate, the cloud and on-premises targets, and the monitoring layer, showing where the decision gates the deployment step on each side independently.

3.4.5 Execution backbone

All external commands (CDK, Ansible, and the scanners) run through a single subprocess helper in `exec_code.py`. The helper merges standard

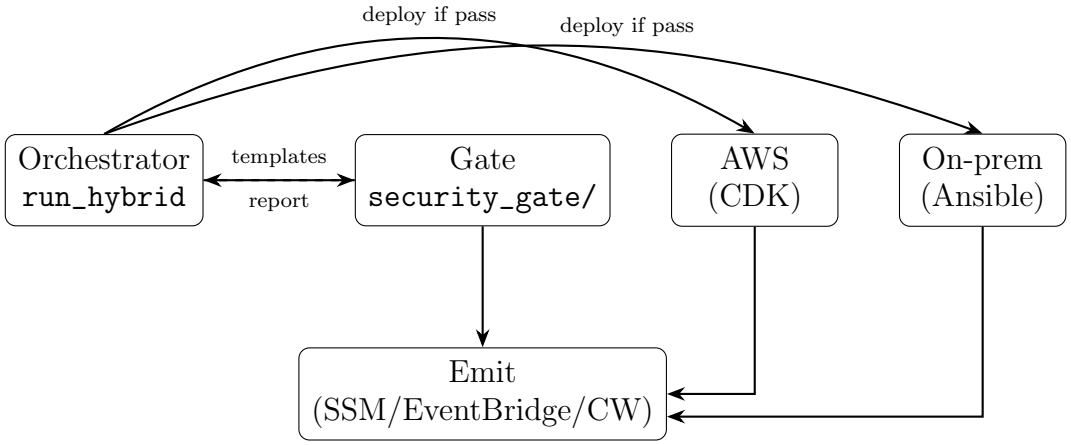


Figure 3.8: Hybrid pipeline interaction sequence. The orchestrator gates both sides with the same engine, deploys each side only on an accepting decision, and emits telemetry.

output and standard error to preserve the ordering of events, optionally streams output line by line for live rendering in the console, and returns the stable `{return_code, output}` contract that every caller relies on. Concentrating process execution in one place keeps the output-capture and error-propagation behaviour consistent across the whole system.

3.4.6 Operator console and multi-project registry

The Streamlit console is a thin orchestration layer over the command-line entry points: every privileged action is delegated to a subprocess with credentials injected through the environment, never as command-line arguments. The console presents the workflow as a sequence of tabs — settings, generation, gate review, plan/diff, deploy, and monitor — and renders the gate report, the decision, and the live monitoring reads. A multi-project registry allows several independent projects to coexist: the default project keeps a flat log layout for backward compatibility, while any other project owns a separate artifact directory so that gate reports, approvals, rejections, and monitoring history never mix between projects. The active project's log directory is exported to the subprocesses through

an environment variable, which is how per-project isolation is enforced end to end.

Chapter 4

Implementation and Evaluation

4.1 Implementation Environment

The proposed SysSecOps process was implemented as a Python-based prototype operating across a hybrid cloud environment. The environment comprised a public-cloud domain and a private infrastructure domain connected through a common orchestration and security assessment workflow. The public-cloud domain provided programmable infrastructure, deployment, compliance information, and operational telemetry, whereas the private domain represented Linux-based on-premises systems managed through infrastructure automation. This arrangement enabled the same operational process and risk evaluation mechanism described in Chapter 3 to be exercised across heterogeneous infrastructure targets.

The implementation environment was organised into four functional groups: prototype execution, infrastructure provisioning, security assessment, and operational monitoring. Table 4.1 summarises the technologies used to realise each group. These technologies are implementation choices for evaluating the proposed process; the process itself is not restricted to these particular products.

Table 4.1: Technologies used in the implementation environment.

Environment group	**Technology**	**Role in the prototype**
Prototype execution	Python, command-line interface, and Streamlit	Implemented the process logic, pipeline entry points, and browser-based operator console.
AI-assisted generation	Amazon Bedrock and OpenRouter	Provided interchangeable language-model backends for generating and regenerating infrastructure code.
Public infrastructure	AWS CDK and AWS CloudFormation	Defined, synthesized, compared, and deployed resources in the public-cloud domain.
Private infrastructure	Ansible and Linux nodes	Represented and configured the on-premises domain of the hybrid environment.
Security assessment	Checkov, `cfn-lint`, `ansible-lint`, and a secret-detection component	Examined infrastructure artifacts for syntax errors, misconfigurations, insecure practices, and exposed secrets.
Contextual risk inputs	Infracost and AWS Config	Supplied estimated cost information and the live compliance state used by the risk-scoring mechanism.
Code-risk analysis	Bandit, Semgrep, and a logistic-regression model	Extracted static analysis features and estimated the probability that generated Python code contained detector-visible insecurity.
Monitoring and audit	AWS Systems Manager, Amazon CloudWatch, AWS CloudTrail, Amazon EventBridge, AWS Lambda, and Amazon SNS	Collected operational state, recorded activity, distributed events, and supported notification and remediation actions.

4.1.1 Prototype Execution Environment

Python was used as the principal implementation language for the prototype. The pipeline was accessible through command-line entry points and a Streamlit operator console. Both interfaces invoked the same orchestration logic, ensuring that an operation performed through the graphical

interface followed the same generation, assessment, approval, and deployment sequence as an operation initiated from the command line. External programs were invoked through a shared process-execution layer, which captured their return status and output in a consistent form for subsequent processing.

AI-assisted infrastructure generation was provided through interchangeable model providers. Amazon Bedrock represented a managed cloud-based model service, while OpenRouter provided an alternative hosted inference interface. The provider was selected through configuration without changing the remaining pipeline. Generated artifacts, synthesis output, scanner findings, risk components, and gate decisions were retained for each run, allowing an unsuccessful generation attempt to be examined and used as feedback for a subsequent attempt.

4.1.2 Hybrid Infrastructure Environment

The public part of the environment was hosted on Amazon Web Services. Infrastructure was expressed as an AWS Cloud Development Kit application written in Python and synthesized into AWS CloudFormation templates. The synthesized template, rather than only the high-level source code, was used as the principal public-cloud assessment artifact. This allowed the security gate to inspect the resource definitions and configuration values that would be submitted for deployment. Representative resources supported by the prototype included networking, compute, storage, and database services within a virtual private cloud.

The private part of the environment consisted of Linux nodes representing on-premises infrastructure. Their desired configuration was expressed through Ansible inventories and playbooks. Before execution, the playbooks were subjected to syntax and static analysis checks. Where operational monitoring across both domains was required, the private nodes were registered as managed instances through AWS Systems Manager hy-

brid activation. This provided a common management path for public-cloud instances and private Linux nodes while preserving their distinct deployment mechanisms.

Credentials and other sensitive configuration values were supplied to the prototype through the execution environment rather than embedded in generated infrastructure code or command-line examples. Deployment remained conditional on the security-gate result: an accepting result allowed the relevant provisioning mechanism to proceed, a review result required explicit approval, and a rejection prevented deployment.

4.1.3 Security Assessment and Risk Environment

The security assessment environment combined several complementary sources rather than depending on a single scanner. Checkov and `cfn-lint` evaluated synthesized CloudFormation templates, while `ansible-lint` evaluated the private-side playbooks. A built-in pattern-based component checked the submitted artifacts for embedded secrets. Their outputs were converted into the common finding representation defined by the proposed methodology before deduplication and scoring.

Two additional services supplied operational context to the assessment. Infracost estimated the financial effect of the proposed infrastructure, and AWS Config provided the compliance state of resources already operating in the public-cloud environment. Consequently, the gate decision reflected not only static misconfiguration findings but also cost and live compliance information.

The code-risk component was implemented as a logistic-regression classifier. Bandit and Semgrep were used to extract severity, confidence, and finding-count features from Python files. The trained classifier converted these features into a probability of detector-visible insecurity, which was then incorporated into the unified risk score. The persisted model and feature scaler were reused during evaluation so that inference followed the

same feature transformation as model training.

Each external assessment component was accessed through an adapter. An unavailable scanner, missing credential, or unsupported input was recorded as a component status instead of terminating the complete workflow. The resulting report therefore contained both the gate decision and the execution status of its contributing assessment sources, making the conditions under which each result was obtained explicit.

4.1.4 Monitoring and Audit Environment

After an accepted deployment, the environment used AWS Systems Manager to obtain managed instance information and support operational actions across the hybrid targets. Amazon CloudWatch collected gate metrics and operational observations, while AWS CloudTrail recorded public-cloud API activity for audit purposes. Amazon EventBridge distributed compliance and deployment events to the operational workflow. Event-driven processing and notification were supported through AWS Lambda and Amazon SNS, respectively.

The monitoring environment consumed the same target identifiers and gate records produced during assessment and deployment. This maintained traceability between a proposed infrastructure change, its security decision, the resulting deployment, and its subsequent operational state. It also provided the implementation basis for evaluating whether the proposed process could integrate pre-deployment assessment with post-deployment monitoring across the public and private domains.

4.2 Evaluation Methodology

The proposed SysSecOps process was evaluated through a controlled prototype case study comprising three complementary investigations. The first investigation examined the operational feasibility and execution reli-

ability of the complete pipeline. The second examined the applicability of the process to an existing on-premises virtual machine. The third examined the performance of the machine-learning component and the correctness of the unified risk-scoring mechanism. This design enabled the evaluation to address the operation of the prototype, its applicability within a hybrid environment, and its ability to transform heterogeneous security evidence into a reproducible deployment decision.

The evaluation focused on observable system behaviour rather than on the capabilities of any individual implementation tool. Evidence was collected from pipeline execution records, synthesized infrastructure artifacts, scanner outputs, risk reports, deployment records, target-state observations, and classifier predictions. The same scoring rules and decision thresholds defined in Chapter 3 were applied throughout the evaluation.

4.2.1 Research Questions

The evaluation was guided by the following research questions:

RQ1. Decision conformance and enforcement correctness: When infrastructure of a known risk band is submitted, does the prototype reach the decision that band warrants, and does that decision actually govern what happens next — such that a *reject* never deploys, a *review* never deploys without a recorded approval, and every run leaves a machine-readable account of itself, including when an optional assessment component or credential is unavailable?

RQ2. Cross-boundary enforcement equivalence: When the same class of weakness is expressed once as public-cloud infrastructure code and once as on-premises configuration, does the shared risk engine *detect* it on both sides of the hybrid boundary, and does the on-premises path attain the same operational checkpoints — reachability, validation, gating, dry run, apply, post-apply verification, and

repeated-run idempotency — as the cloud path?

RQ3. Unified risk-evaluation capability: To what extent can the proposed risk-scoring mechanism integrate findings from multiple security tools and a logistic-regression code-risk estimate to distinguish insecure code and produce consistent, interpretable deployment decisions?

Two aspects of this formulation need to be reviewed. First, RQ1 deliberately does not ask whether runs *complete*. A gate that never blocks anything completes every run, so a completion rate cannot by itself distinguish a working security control from an inert one. Conformance to a declared expectation, and enforcement of the resulting decision, are the properties that carry evidential weight; completion is reported only as a supporting measure. Second, RQ2 is stated as a question about detection rather than about decision equality. An earlier formulation asked only whether the process *could be applied* to an on-premises node, which the prototype's existence already answers. A later one asked whether both sides reach the same decision band — but the two sides are not the same artifact, and demanding identical bands would assert an equivalence between a firewall rule and a security group that does not hold, while being satisfiable by simply retuning weights. What can fail, and what matters operationally, is whether a weakness visible on one side is invisible on the other. Section 4.3.2 reports a case where it was.

Table 4.2 presents the relationship between the research questions, the corresponding evaluation methods, and the principal measurements.

4.2.2 Evaluation Design

The unit of analysis for RQ1 was one complete pipeline run. A run began when an infrastructure request or existing project was submitted to the prototype and ended when the process produced a persisted terminal

Table 4.2: Relationship between the research questions and evaluation measures.

RQ	Evaluation method	Collected evidence	Measures
RQ1	Replicated execution of pre-registered scenarios spanning three risk bands on both branches, plus scanner-removal scenarios	Gate reports, per-stage timing records, enforcement outcomes, and campaign run records	Decision conformance, confusion matrix, enforcement-invariant satisfaction, degradation behaviour, and gate overhead
RQ2	Paired cloud/on-premises fixtures expressing the same weakness class, and checkpoint attainment on a Linux node reached over SSH	Per-branch finding categories, paired gate scores, connectivity results, dry-run and apply records, verification results, and idempotency counters	Detection coverage per weakness class, checkpoint attainment rate, and failure-class distribution
RQ3	Held-out classifier evaluation and controlled risk-decision cases	True labels, predicted probabilities, confusion matrix, component scores, deduplicated findings, and gate decisions	Accuracy, precision, recall, F1-score, ROC–AUC, score correctness, and decision consistency

decision. Where the decision permitted deployment, the run additionally included deployment and post-deployment status collection. For RQ2, the unit of analysis was one pipeline-controlled Ansible operation against an existing virtual machine. For RQ3, an individual Python file constituted the unit of analysis for classifier evaluation, while one controlled combination of risk inputs constituted the unit of analysis for gate-decision evaluation.

The evaluation varied three principal conditions: the risk characteristics of the submitted infrastructure, the deployment target, and the availability of optional assessment components. The observed dependent variables included completion status, stage outcome, gate decision, deployment outcome, resulting target state, predicted code-risk probability, and agreement between independently calculated and reported risk scores.

A security rejection was not classified as a pipeline failure. A run that identified an unacceptable change, produced a valid *reject* decision,

and prevented deployment represented correct execution of the security process. An unexpected failure was defined as an unhandled exception, premature pipeline termination, absent or malformed decision report, incorrect transition between stages, or deployment that violated the gate and approval policy.

4.2.3 Pre-Registered Evaluation Campaign

The measurements reported in Section 4.3 were not gathered by reading back the operational logs the prototype happened to accumulate during development. Such logs are a convenience sample: they over-represent whatever was being debugged at the time, they contain runs of incomplete projects, and the analyst chooses after the fact which of them to count. To avoid this, the evaluation was restructured around a campaign whose scenarios, expected outcomes, and replicate counts were declared in a version-controlled specification *before* any of the reported runs were executed, and whose results were then computed from the persisted artifacts by a single analysis program.

Calibrated Fixtures

Thirty infrastructure fixtures were constructed. Six are *band fixtures*: for each branch — AWS CDK for the cloud path and Ansible for the on-premises path — one fixture was authored to sit in each of the *pass*, *review*, and *reject* bands. The remaining twenty-four form twelve *matched pairs*, each pair expressing the same thing twice, once in CDK and once in Ansible. Eleven pairs express one catalogued weakness class each: unrestricted network ingress, a plaintext credential, over-privileged access, unencrypted storage, insufficient logging, missing authentication, incorrect permissions, externally accessible files, unbounded resources, cleartext transmission, and execution with unnecessary privileges. The twelfth pair is the *specificity control*, a compliant stack and a compliant playbook that

express no weakness at all and against which every detection matcher must remain silent.

Each fixture carries a declared band in `expectations.yaml` together with the finding breakdown that produces it. A calibration utility re-scores every fixture and fails if any has drifted outside its declared band, so a scanner upgrade that silently changes the arithmetic is caught rather than absorbed. An important methodological constraint applies to the matched pairs: their declared bands are *descriptive*, recorded from measurement, and were not adjusted to bring the two branches into line. Tuning the fixtures or the weights until both sides returned the same band would have manufactured a result while leaving the question RQ2 untouched. For the same reason the bands recorded for the coverage pairs are those measured in the baseline arm and were deliberately left unchanged when the detection rules were added; the remediated arm consequently reports the affected fixtures as non-conformant against them. That divergence is the effect being measured, and updating the bands to match would erase it.

Scenario Matrix

The specification in `evaluation/scenarios.yaml` declares 48 scenarios totalling 154 offline runs, organised in six groups. Eight *conformance* scenarios, replicated five times each, submit the six band fixtures and additionally exercise the *review* band twice per branch — once withheld and once with an operator approval supplied — so that the approval gate is observed in both states. Ten *degradation* scenarios, replicated three times each, remove one scanner from an otherwise identical run; each declares the decision that the remaining evidence should support, derived arithmetically from the calibrated breakdown, so the scenario tests a prediction rather than merely recording whatever occurs. Eight *parity* scenarios, replicated three times each, submit the four original pairs, and four *parity baseline* scenarios repeat the on-premises half of each pair with the seman-

tic configuration rules disabled; these are retained as a regression check that the rules are the component responsible for the on-premises findings, and are not reported separately. Sixteen *coverage* scenarios, replicated three times each, submit the seven additional matched pairs derived from the external weakness catalogue together with the two specificity controls; these and the eight parity scenarios form the coverage arm of 24 scenarios and 72 runs, which is executed twice — once at the frozen gate commit and once after the detection rules were added — yielding the baseline and remediated measurements. The conformance, degradation and parity groups together form the 106-run batch on which the RQ1 results are reported; the eight parity scenarios belong to both that batch and the coverage arm, which is why the two totals overlap. Two further scenarios, replicated three times each, exercise real deployment against a live node; they are skipped unless the `target-host` capability is explicitly granted, which keeps the offline campaign self-contained and reproducible without any external dependency. They are reported separately as the live-target arm, run against a disposable Multipass virtual machine provisioned by `scripts/setup_multipass_target.sh`.

Controlling Non-Determinism

Two components make a run's score depend on the day it executes: the cost estimator prices against a live feed, and the cloud configuration check reports live account state. During pilot execution this was not hypothetical — a *reject* fixture scored 119 against a calibrated 114 because live pricing contributed an additional cost band. The registered campaign therefore disables both components by default and records that fact in each run's manifest, so that a reported score is reproducible from the repository alone. A separate, explicitly non-deterministic arm can be requested when the interaction with live pricing is itself of interest.

Recorded Artifacts and Analysis

Every run writes a self-describing report containing the run identifier, wall-clock duration, terminal status, the scenario and campaign it belongs to, the conditions in force — deployment enabled, approval supplied, scanners disabled, and the numeric thresholds — the provenance of the environment, and a per-stage record giving each stage's outcome, duration, and, on failure, a classified failure reason. Provenance capture includes the repository commit, whether the working tree was dirty, the interpreter version, the host, and the resolved version of every external tool, so that a table can be traced to the exact software state that produced it. The campaign runner appends one row per run to a line-delimited record and copies each report into the campaign directory.

All reported statistics are computed from those persisted records alone; the analysis program never re-executes the pipeline. Proportions are reported as a count over the number of runs to which the measure applied, rather than as a percentage alone. The denominator is carried alongside every rate because several of them are small — some invariants and checkpoints are exercised by ten runs or fewer — and a rate of 1.00 over ten runs supports a much weaker claim than the same rate over 106. No confidence intervals are reported: the campaign's replicate counts were chosen to cover the declared scenarios rather than to estimate failure frequencies, and an interval computed over three or ten runs would suggest a precision the design does not provide. The run counts are stated instead, so that the strength of each result is visible directly. Durations are reported as medians with inter-quartile ranges rather than means, because pipeline stage times are right-skewed by occasional tool start-up costs.

4.2.4 End-to-End Pipeline Evaluation

RQ1 was investigated by executing the prototype across representative scenarios that exercised the principal paths through the operational

workflow. Each run was observed at the following checkpoints:

1. acceptance or generation of the infrastructure source;
2. synthesis of the public-cloud template or syntax validation of the private-side playbook;
3. execution of the configured assessment components;
4. normalisation and deduplication of the collected findings;
5. calculation and persistence of the component scores, total score, and gate decision;
6. enforcement of the deployment and approval policy; and
7. publication of the decision and deployed-target state to the monitoring and audit layer.

Four scenario classes were included. A low-risk change exercised the *pass* path, a moderate-risk change exercised the manual *review* path, and a deliberately insecure change exercised the *reject* path. A degraded-execution scenario was also included by making an optional assessment dependency or credential unavailable. This scenario examined whether the corresponding component returned an explicit unavailable or skipped status without causing an unhandled termination of the complete pipeline. The degraded scenario was analysed separately because successful continuation with a missing assessment source provides less security evidence than a fully assessed run.

For n pipeline runs, the end-to-end completion rate was calculated as

$$\text{Completion Rate} = \frac{n_{\text{terminal}}}{n} \times 100\%, \tag{4.1}$$

where n_{terminal} denotes the number of runs that reached a valid and persisted terminal decision without an unexpected failure. This rate is

reported as a supporting measure only. The primary measure for RQ1 is *decision conformance*, the proportion of runs whose observed decision equals the decision declared for that scenario in advance:

$$\text{Conformance} = \frac{|\{r : d_{\text{obs}}(r) = d_{\text{exp}}(r)\}|}{n} \times 100\%, \tag{4.2}$$

where $d_{\text{obs}}(r)$ and $d_{\text{exp}}(r)$ denote the observed and pre-declared decision for run r. Conformance is reported both in aggregate and as a confusion matrix over the three decision bands, so that a systematic bias towards permissiveness or towards over-blocking would be visible rather than averaged away.

Stage-level completion was reported in addition to the aggregate rate to identify whether failures occurred during generation, validation, assessment, scoring, deployment, or monitoring. Enforcement was then evaluated against a set of invariants that must hold for every run regardless of its decision: a *reject* decision is never followed by a deployment; a *review* decision is followed by a deployment only when an approval record exists; every run persists a decision report; every non-passing decision persists a corresponding rejection or approval record; and every report carries the provenance needed to reproduce it. Each invariant is reported as a satisfaction rate over the runs to which it applies, so a single violation is visible rather than diluted by the aggregate.

Finally, gate overhead was measured, since a control that is correct but prohibitively slow will not survive contact with a real delivery process. The duration of the gate stage and its share of total run wall-clock time were recorded per run and reported as medians with inter-quartile ranges.

4.2.5 Cross-Boundary Equivalence Evaluation

RQ2 examined whether the two sides of the hybrid boundary are governed alike. This has two parts, which are measured separately: whether the on-premises path can attain the same operational checkpoints as the

cloud path, and whether the shared risk engine detects the same class of weakness when it is expressed on either side.

The on-premises target was a Linux virtual machine represented in an Ansible inventory and configured through an Ansible playbook. For the live-target arm it was a disposable Ubuntu 22.04 instance provisioned through Multipass and reached over SSH with a purpose-generated key pair injected at first boot; the hybrid case study of Section 3.4 instead reaches an equivalent private-side host over a mesh-VPN connection between the two domains. The distinction is one of transport only — in both cases the orchestrator sees an inventory entry it must reach, authenticate to, and converge — and using a disposable instance keeps the measurement reproducible by anyone with the provisioning script. The same risk engine, scoring rules, and decision thresholds applied to public-cloud changes governed this private-side target.

The inventory is supplied as a file declaring the host in the `appservers` group rather than as a bare hostname on the command line. This is a correctness requirement, not a preference: the fixture playbooks target that group, and `ansible-playbook` exits successfully with an empty `PLAY RECAP` when no host matches a play, so a group mismatch would record a deployment that applied nothing as a successful apply.

Checkpoint Attainment

Seven operational checkpoints were defined, corresponding to the stages a change must pass to move from submission to a verified deployed state: target reachability, syntax validation, security gating, dry run, apply, post-apply verification, and repeated-run idempotency. For each run, a checkpoint was recorded as attained, failed, or not attempted. Attainment rates were computed over attempted checkpoints only. This distinction matters: when the gate returns *reject*, the dry-run checkpoint is never attempted, and counting that as a failure would penalise the system for behaving correctly.

The evaluation procedure comprised six stages. First, network reachability and authenticated Ansible access to the target were verified. Second, the inventory and playbook were submitted to syntax validation and static security assessment. Third, the resulting findings were normalised and processed by the same risk engine and decision thresholds used for public-cloud changes. Fourth, playbook execution was permitted only when the gate decision and approval state satisfied the deployment policy. Fifth, the requested configuration state was verified directly on the virtual machine. Finally, the target identity, gate decision, execution result, and post-execution state were recorded for operational traceability.

Successful integration required all of the following conditions: authenticated access to the existing target, successful analysis of the playbook, production of a valid gate decision, enforcement of that decision before execution, successful application of an allowed configuration, and verification of the requested target state. Where the playbook described an idempotent operation, a subsequent execution was used to determine whether the desired state remained stable without unintended configuration changes.

Observed failures were classified as connectivity, authentication, validation, assessment, gate-enforcement, configuration-execution, or target-verification failures. This classification separated failures caused by the external infrastructure from those caused by the implementation of the proposed process.

Detection Coverage

Checkpoint attainment establishes that the on-premises path *runs* the same workflow. It does not establish that the workflow *sees* the same things. Testing the latter requires a set of weakness classes to test over, and the choice of that set is the point at which such an evaluation is most easily compromised. A set chosen by the author cannot distinguish a gate that covers the space of deployment-time weaknesses from a gate whose author chose the weaknesses it already covers. The class list was therefore

derived from an external catalogue rather than written for the purpose.

Derivation of the class list. The catalogue is MITRE's Common Weakness Enumeration view CWE-1008, *Architectural Concepts*, which organises weaknesses by the security principle they violate rather than by language or platform [27]. That organisation is what makes it usable here: a view indexed by principle yields classes that can be expressed on either side of the hybrid boundary, whereas a view indexed by technology would not. Version 4.20 of the view was retrieved as the published XML export, and its archive digest, version, and release date are recorded so that the derivation can be repeated against the same source.

The view contains 12 categories spanning 223 member weaknesses. Three filters were applied mechanically. The first retains only entries that are members of a CWE-1008 category. The second retains only entries whose abstraction is *Base* or *Class*, discarding variants too specific to have an expression in both technologies and pillars too abstract to be tested at all. The third retains only entries whose declared Modes of Introduction include at least one of *Operation*, *System Configuration*, *Installation*, or *Bundling* — that is, weaknesses that a deployment artifact can introduce, as distinct from those introduced only while writing application code. A gate that inspects infrastructure definitions cannot be asked to see the latter, and including them would understate coverage by counting weaknesses outside the mechanism's remit.

The filters reduce 223 members to 38 candidates spanning seven categories; five categories are eliminated entirely, and the reason for each elimination is recorded rather than left implicit, because a category with no deployment-time expression bounds what any pre-deployment gate can be asked to see. From the 38 candidates, 11 classes were retained as those expressible as a matched pair in both a CDK application and an Ansible playbook; the 27 candidates not retained are listed individually with the reason for exclusion. The resulting catalogue is a pre-registration: it fixes

the classes, their CWE identifiers, and the matcher that decides whether a report names each class, before any of them was measured.

Fixtures and the specificity control. Each of the 11 classes was authored twice — once as CDK infrastructure code and once as an Ansible playbook — forming a matched pair in which the intended weakness is held constant and only the boundary varies. Each fixture carries one weakness and is otherwise deliberately unremarkable, so that a difference in the gate's report is attributable to the weakness rather than to incidental differences between the two artifacts.

Crediting detection from the weakness fixture alone would reward a matcher that responds to everything. Each branch therefore also submits a *control* fixture carrying none of the catalogued weaknesses, and a class is credited only if its matcher fires on the weakness fixture *and* stays silent on the control for the same branch. Where the control was never examined, the class is recorded as not measured rather than as detected, so that an absent control cannot be mistaken for a silent one.

The controls were not the ones originally intended. The RQ1 *pass* fixtures were used first, and the cloud one proved unusable: it provisions a bucket that receives S3 access logs, and such a bucket cannot log to itself and must carry a log-delivery ACL, so the report named both access logging and a public ACL no matter how the fixture was written. Two classes therefore had a control that could not discriminate. Purpose-built controls were substituted — on the cloud side a key, an encrypted log group, and an encrypted queue with a dead-letter queue; on the on-premises side a playbook that installs a service under a dedicated account with stated limits — and both were verified to produce no findings at all before the measurement was taken. The original measurement is retained rather than discarded.

Two arms. The gate was frozen at a recorded commit before the first fixture was authored, and the *baseline* arm was measured at that commit. Detection rules written in response to the baseline result were then implemented, and the *remediated* arm was measured over the identical scenario set. Reporting both, with the frozen commit recorded, states plainly which capability preceded the measurement and which followed it.

Two constraints keep the second arm from collapsing into fitting. First, the matchers in the catalogue are frozen at their pre-registered form even where they are known to under-credit, so a class is earned by emitting the pre-registered category rather than by widening the pattern that recognises it after seeing what the pattern missed. Second, each new rule is accompanied by unit tests that express the same weakness through a different module or resource type from the one the fixture uses, and by a test asserting that the rule stays silent on the secure form of the same construct. A rule that passed only against its own fixture would satisfy neither.

What is measured, and what is not. The measured quantity is coverage, not agreement. For each pair and each branch, the persisted gate report is examined for a finding whose category names the weakness the fixture encodes; a generic style or hygiene finding that happens to touch the same artifact is not credited, since a report that names the wrong problem gives an operator no signal that the real one exists. Alongside the per-class result, the classes are cross-tabulated by which branches saw them, so that a gap is located on a specific side of the boundary rather than averaged away.

Decision agreement was deliberately *not* adopted as the measure. The two sides of a pair are not the same artifact — a host firewall rule and a security group differ in blast radius, in reversibility, and in how many distinct controls they can violate — so requiring identical bands would assert an equivalence that does not hold, and could be satisfied by adjusting weights rather than by improving detection. Where a branch fails to detect

a weakness, the finding breakdown of both sides is reported so that the cause can be attributed to a specific absence of capability rather than left as an unexplained numeric difference.

4.2.6 Risk-Scoring Evaluation

Dataset Construction

The evaluation corpus contained a positive class and a negative reference class. The positive class was obtained from the SecurityEval dataset, which contains Python vulnerability examples associated with Common Weakness Enumeration categories and was developed for evaluating the security of machine-learning-based code-generation techniques [28]. Each selected vulnerability example was stored as an individual Python file and assigned label 1.

The negative class was sampled from the source trees of maintained open-source Python projects, including `click`, `rich`, and `typer`. Each selected file was assigned label 0. These files constituted a *reference secure* class rather than a formally verified secure class. Their use in maintained software projects provided practical negative examples, but did not establish the absence of every possible vulnerability.

To improve comparability between the two classes, package initialisers and test files were excluded. Files containing fewer than 20 or more than 700 non-blank, non-comment lines were also excluded. These criteria reduced the influence of trivial snippets, tests, and unusually large files on the learned model. The number of samples, class distribution, and originating project or dataset were retained as dataset metadata. The final corpus was balanced as far as practical to limit majority-class bias.

Feature Extraction and Model Training

Each file was analysed independently using Bandit and Semgrep. The findings were reduced to an eleven-dimensional feature vector comprising Bandit severity counts, Bandit confidence counts, Semgrep severity counts, and the total number of findings returned by each analyser 3.3.4.

The corpus was divided into training and test partitions using a 80/20 split. A feature scaler was fitted exclusively on the training partition and was then applied unchanged to the held-out test partition. The logistic-regression classifier was trained with balanced class weights, and its output for each file was the estimated probability of membership in the insecure class.

Classifier Evaluation Measures

Classifier performance was assessed using the confusion matrix, accuracy, precision, recall, F1-score, and the area under the receiver operating characteristic curve (ROC–AUC). Let TP, TN, FP, and FN denote the numbers of true positives, true negatives, false positives, and false negatives, respectively. The principal measures were calculated as

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{4.3}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{4.4}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \tag{4.5}$$

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{4.6}$$

Accuracy represented overall predictive correctness, whereas precision measured the proportion of predicted insecure files that belonged to the positive class. Recall measured the proportion of labelled insecure files

identified by the model. Recall was particularly relevant to the gate because a false-negative result reduced the learned risk contribution assigned to insecure code. The F1-score summarised the balance between precision and recall, while ROC–AUC measured the ability of the model to rank positive samples above negative samples across probability thresholds.

A scanner-only rule provided the baseline for comparison with the logistic-regression classifier. The baseline classified a file as insecure when at least one configured security finding was reported. This comparison determined whether learning a relationship among the scanner features provided discriminatory value beyond treating the presence of any finding as a positive classification.

4.3 Evaluation Results

4.3.1 RQ1: Decision Conformance and Enforcement Correctness

Decision Conformance

Across all 106 runs, the observed gate decision equalled the decision declared in advance in every case, giving a decision conformance of 100% (106/106) by Equation 4.2. Conformance was 1.0 for each of the 30 scenarios individually, so the aggregate does not conceal a scenario that failed consistently while others compensated. Replicated runs of the same scenario produced identical scores in every case — for instance, all five replicates of `cdk-reject-block` scored exactly 114 and all five of `ans-reject-block` scored exactly 155 — confirming that the scoring pipeline is deterministic once the two live-state components are disabled.

Table 4.3 presents the confusion matrix over the three decision bands. This matters more than the aggregate proportion, because a gate can achieve a high overall agreement rate while being systematically permis-

sive — passing changes it should have held — and that bias would be invisible in a single percentage. Here there is neither a permissive nor an over-blocking tendency to report.

Table 4.3: Expected against observed gate decision, all 106 campaign runs.

Branch	**Expected**	**Obs. pass**	**Obs. review**	**Obs. reject**	n
Ansible	Pass	26	0	0	26
Ansible	Review	0	25	0	25
Ansible	Reject	0	0	8	8
CDK	Pass	11	0	0	11
CDK	Review	0	28	0	28
CDK	Reject	0	0	8	8

This result should be read for what it establishes and what it does not. The fixtures were authored by the same work that built the gate, so conformance demonstrates that the implemented system behaves as its specification requires on inputs whose correct treatment is known — it does not demonstrate that the specification assigns the right band to arbitrary real-world infrastructure. That stronger claim would require fixtures authored independently, and is recorded as a limitation in Section 5.3.

Enforcement Correctness

Conformance concerns the decision the gate reaches. Enforcement concerns whether that decision actually constrains what happens next, which is the property that distinguishes a working control from an inert one. Each invariant in Table 4.4 is reported over the runs to which it applies, so that a single violation would be visible rather than diluted.

Every invariant held on every applicable run. The denominators matter as much as the rates, and they are not comparable to one another. Summary persistence and scanner-status reporting were exercised on all 106 runs; the approval-record invariant was exercised on ten, and the two rejection invariants on sixteen. A rate of 1.00 over ten runs is a much weaker

Table 4.4: Enforcement invariants over the campaign.

Invariant	n	**Held**	**Rate**
Run produces a persisted summary	106	106	1.00
Gate reports a status for every scanner	106	106	1.00
Unapproved *review* halts	43	43	1.00
Approved *review* writes an approval record	10	10	1.00
Reject blocks deployment	16	16	1.00
Reject writes a rejection record	16	16	1.00

statement than the same rate over 106 — it is consistent with a defect that occurs infrequently enough to have been missed — so the last four rows should be read as evidence that the mechanism works when exercised, not as an estimate of how often it fails.

Behaviour Under Degraded Assurance

Ten scenarios removed a single scanner from an otherwise identical run. In every case the resulting decision matched the decision predicted arithmetically from the calibrated finding breakdown, so the gate degrades predictably rather than erratically. Table 4.5 reports the score deltas.

The most consequential row is the fourth. Removing Checkov from a *review*-band CDK change drops the score from 54 to 9 and moves the decision to *pass*. The gate conformed to its specification — this is precisely the outcome the scenario declared in advance — but the specification permits a change to be released because the evidence that would have held it was never collected. The gate fails *open*. The first row shows the same mechanism short of a release: removing Checkov from a *reject* change costs 50 points and downgrades it to *review*, and removing the secret scanner from the Ansible *reject* fixture costs 80 of its 155 points. In each case the loss of a single assurance source materially weakens the verdict while the run still reports a clean terminal status.

The two ML risk model rows isolate a different property: what that

Table 4.5: Score and decision change when one assurance source is removed. Each row is the median of three replicates, which were identical.

Component removed	**Full**	**Degr.**	**Δ**	**Decision change**
Checkov (from *reject*)	114	64	50	reject → review
ML risk model (from *reject*)	115	70	45	reject → review
cfn-lint (from *review*)	54	49	5	review → review
Checkov (from *review*)	54	9	45	review → **pass**
ML risk model (from *review*)	63	25	38	review → review
Secret scan (from *reject*)	155	75	80	reject → review
ansible-lint (from *reject*)	155	135	20	reject → reject
ansible-lint (from *review*)	55	25	30	review → review
Secret scan (from *review*)	55	35	20	review → review
Checkov (from *pass*)	0	0	0	pass → pass

component contributes that nothing else on its branch does. Removing it from the fixture whose weakness sits in bundled handler source costs 45 of 115 points and returns a *reject* to *review*. That weakness never reaches the synthesised template, so no infrastructure scanner observes it at all, and the ML risk model is the only source that scores it — it is what carries the stack across the reject threshold. Removing it from the plaintext-secret fixture costs 38 of 63. The cloud branch has no secret scanner, so again no other component responds to the weakness the fixture encodes. Both rows show the component carrying decision weight no other assurance source on that branch supplies, and both show that weight disappearing silently when it is unavailable.

This is a design deficiency of the scoring model, not an implementation defect, and it is inherent to any purely additive score: an absent scanner and a scanner that found nothing contribute identically. The scanner-status record makes the degradation visible to an auditor after the fact, but nothing in the decision function acts on it. Section 5.3 returns to this point, and Chapter 6 proposes an assurance penalty as the remedy.

Gate Overhead

A control that is correct but prohibitively slow will not survive contact with a real delivery process, so gate duration and its share of run wall-clock time were recorded per run. Table 4.6 reports medians with inter-quartile ranges, since stage times are right-skewed by tool start-up costs.

Table 4.6: Stage duration and gate share of total run time.

Branch	**Stage**	n	**Median**	**IQR (p25–p75)**
CDK	Synthesis	47	13.87 s	11.91–16.81 s
CDK	Difference	47	14.93 s	12.99–16.23 s
CDK	Security gate	47	102.50 s	101.14–102.73 s
CDK	Gate share of run	47	69.35%	67.55–70.51%
Ansible	Syntax validation	59	0.52 s	0.52–0.53 s
Ansible	Dry run	31	5.46 s	5.46–5.46 s
Ansible	Security gate	59	7.04 s	4.89–7.29 s
Ansible	Gate share of run	59	24.80%	20.54–29.02%

The two branches differ by more than an order of magnitude: the cloud gate takes a median of 102.50 seconds and consumes roughly seven-tenths of total run time, whereas the on-premises gate takes 7.04 seconds and consumes about a quarter. The cloud figure is dominated by Checkov, which is invoked once per synthesised template file rather than once per directory. In absolute terms both are acceptable for a pre-deployment check — under two minutes on the slower branch — but the cloud gate would become the dominant cost of a pipeline whose other stages were optimised, and it scales with template count rather than with the size of the change.

4.3.2 RQ2: Cross-Boundary Enforcement Equivalence

Checkpoint Attainment

Table 4.7 reports the seven operational checkpoints over the 65 Ansible-branch runs, separated into the 59 runs of the offline campaign and the 6 runs of the live-target arm. Attainment is computed over attempted checkpoints only: when the gate returns *reject*, the dry run is never attempted, and counting that as a failure would penalise the system for behaving correctly.

The offline campaign is deliberately hermetic, so the two scenarios that touch a real node are skipped unless the `target-host` capability is granted at invocation. Granting it requires a host to grant it against. The live-target arm therefore provisions a disposable Ubuntu 22.04 virtual machine through Multipass, injects a dedicated SSH key pair at first boot, and points the orchestrator at an inventory file naming that machine in the `appservers` group. Two scenarios run against it: `ans-pass-deploy`, which carries the clean fixture through apply, post-apply verification and a repeated apply; and `ans-reject-deploy-blocked`, which requests deployment of the rejected fixture and must not get one. All six runs were decision- and status-conformant.

No run in either arm recorded a failure of any attempted checkpoint; all 65 runs were classified in the failure class `none`. Three observations from the live arm carry more weight than the rates themselves, because each is a distinct kind of evidence that the offline campaign structurally could not produce.

First, apply is not merely a zero exit code. Post-apply verification is performed by a separate playbook that reads the resulting state back off the host and asserts against it: that the service directory exists with mode `0750` owned by the service account, that the configuration file exists with mode `0640`, that the account exists with a non-interactive shell, and that the deployed configuration binds the service to the loopback interface. Its

Table 4.7: On-premises checkpoint attainment across 65 Ansible-branch runs.

Checkpoint	**Offline (59)**		**Live target (6)**	
	n	**Attained**	n	**Attained**
Target reachability	0	—	6	6
Syntax validation	59	59	6	6
Security gate	59	59	6	6
Dry run	31	31	3	3
Apply	0	—	3	3
Post-apply verification	0	—	3	3
Repeated-run idempotency	0	—	3	3

expected values are written literally rather than read from the fixture's variables — asserting against the same variables that produced the state would accept a wrong variable value as correct. Nine assertions executed with zero failures on every apply run.

Second, idempotency is measured rather than assumed. Each `ans-pass-deploy` run applies the playbook a second time and parses the resulting `PLAY RECAP`; all three recorded `changed=0`, so the converged state is stable under re-execution.

Third, and most directly relevant to enforcement, the three `ans-reject-deploy-blocked` runs requested deployment explicitly and never received one. Their recorded stage sequences terminate after `ansible.gate`: no dry run, no apply, no verification. The reject decision blocked deployment against a live, reachable, credential-verified host rather than in simulation.

These results should not be over-read, and the run counts are the reason. The last four checkpoints in Table 4.7 rest on three runs each, executed against one host running one operating system image from a fresh boot. Three successes are enough to show that the apply, verification and idempotency machinery works and produces the records it claims to; they

are far too few to characterise how often it fails. The same applies to the enforcement result: three blocked deployments demonstrate that the gate stops a rejected change at a live host, but a defect that manifests only occasionally would not have appeared in three attempts. What the live arm establishes is that the deployment path completes correctly under the conditions tested, not that it is reliable across host heterogeneity, concurrent targets, or partial failures.

Detection Coverage Across the Boundary

Checkpoint attainment shows that the on-premises path *runs* the same workflow. Whether it *sees* the same things is a separate question, and the one on which RQ2 turns.

It is tempting to state that question as decision equality — that the same weakness should receive the same band on both sides — but that is the wrong property to demand. An `iptables` rule is not a security group, and a `sudoers` entry is not an IAM policy. They differ in blast radius, in how easily they are reverted, and in how many independent defects a scanner can legitimately raise against them. Requiring identical bands would assert an equivalence between artifacts that are not equivalent, and it could be satisfied trivially by adjusting weights until the fixtures agreed, which would measure the adjustment rather than the system.

The property that does matter is weaker and more fundamental: a weakness class the gate can see on one side of the boundary should not be *invisible* on the other. A weakness that produces no finding cannot be scored, cannot be routed to review, and cannot be audited afterwards, whatever the thresholds are set to. Detection coverage is therefore measured directly: for each matched pair, did the branch emit a finding whose category names the weakness the fixture encodes? The criterion is applied identically to both branches, and a generic style finding that happens to touch the same artifact does not count as detection.

Two decisions keep that criterion from being self-serving. First, the

eleven weakness classes are not the author's list: they are the classes that survive three mechanical filters applied to the 223 members of the MITRE CWE-1008 *Architectural Concepts* view, whose archive digest is recorded alongside the class list in `evaluation/coverage_catalogue.yaml`. Second, the measurement is taken twice. The *baseline* arm was run with the gate frozen before any fixture had been inspected for whether it would be detected; the *remediated* arm was run after detection rules were written for the gaps the baseline exposed. Each arm is 72 runs — twenty-four scenarios, eleven weakness pairs and one control pair, at three replicates each. The class list, the fixtures and the matchers were identical in both arms, and the baseline was never recomputed after the rules were added. Table 4.8 compiles both arms from the gate reports referenced by each run record.

At the frozen commit the gate named seven of the eleven classes on the cloud branch and five on the on-premises branch. The intervals are wide — eleven classes cannot pin a rate down — and they overlap, so the difference between 7/11 and 5/11 is not itself a finding. What the arm does establish is which specific classes were invisible, and that is not a matter of interval width: a class named in zero of three runs was named in zero of three runs.

The shape of that result matters more than the totals. Table 4.9 cross-tabulates the eleven classes by which branch named them. Only three classes were named on both sides at the frozen commit. Two were named only on-premises, four only in the cloud, and two on neither. A gate that treats the two branches as interchangeable would be wrong about eight of eleven classes.

Reading the baseline misses individually shows that they are not all the same kind of failure. Most are genuine blind spots: no component in that branch inspected the construct at all, so no wording of the criterion would have credited a detection. One is not. On the cloud `incorrect-permissions` fixture the baseline report contained exactly one finding — `CKV_AWS_33`, "Ensure KMS key policy does not contain wildcard (*) principal." The

Table 4.8: Detection of each catalogued weakness class, by branch and by arm. "Base" records whether the baseline arm, at the frozen gate commit, emitted a finding naming the class; the adjacent column gives the categories that named it in the remediated arm.

Weakness	**Cloud branch**		**On-premises branch**	
	Base	**Remediated categories**	**Base**	**Remediated categories**
CWE-778 Insufficient logging	yes	`s3_access_logging`	no	`audit_logging`; `log_retention`
CWE-306 Missing authentication	no	`api_auth_none`	no	`ssh_empty_password`
CWE-269 Improper privilege management	yes	`iam_wildcard`	yes	`iam_wildcard`
CWE-732 Incorrect permissions	no	`permissive_resource_policy`	yes	`file_world_writable`
CWE-668 Exposure to wrong sphere	yes	`sg_ssh_open`	yes	`sg_ssh_open`
CWE-552 Externally accessible files	yes	`s3_public_access_block`	no	`nfs_world_export`
CWE-770 Unbounded resources	yes	`lambda_concurrency`	no	`missing_resource_limit`
CWE-311 Missing encryption	yes	`ebs_encryption`	yes	`storage_encryption`; `storage_mount_encryption`
CWE-319 Cleartext transmission	yes	`CKV_AWS_103`; `CKV_AWS_131`	no	`insecure_transport`
CWE-922 Insecure secret storage	no	— (rule withheld)	yes	`secret_password`; `secret_token`
CWE-250 Unnecessary privileges	no	`privileged_container`; `runs_as_root`	no	`runs_as_root`; `unnecessary_privileges`
Coverage	**7/11**	**10/11**	**5/11**	**11/11**

Table 4.9: The eleven weakness classes cross-tabulated by which branch emitted a finding naming them, in each arm.

Named by	**Baseline**	**Remediated**
Both branches	3	10
Cloud branch only	4	0
On-premises branch only	2	1
Neither branch	2	0
Total	11	11

scanner saw the weakness. The finding did not carry a category naming the class, and the pre-registered matcher for CWE-732 did not recognise Checkov's phrasing, so the run counted as undetected. This is worth stating plainly rather than repairing quietly, because it is the shared-vocabulary problem in miniature: a finding that describes the right weakness in the wrong words is, to every downstream consumer — the deduplicator, the risk score, the audit record, the operator — indistinguishable from no finding at all. The matcher was left frozen and the gate was given a rule that emits `permissive_resource_policy` directly, which is the same remedy the argument recommends: fix the vocabulary at the producer, not at the reader.

The baseline left ten gaps in total — four on the cloud branch and six on-premises — and the remediated arm applied that remedy to nine of them: six rule families were added to the on-premises configuration adapter and three to the cloud template analyser, each emitting a category that was already in the pre-registered list for its class. Coverage rose to 10/11 in the cloud and 11/11 on-premises, and the cross-tabulation collapsed from eight discordant classes to one. Both specificity controls — a compliant CDK stack and a compliant playbook, run in the same campaign — remained silent on every one of the eleven classes in both arms, so the added rules respond to the weakness rather than to the fixture. As a second constraint, each new rule is accompanied by unit tests that express the same weakness

in a different form from the fixture that motivated it, paired with the secure form of the same construct; a rule that fired on both would have bought coverage at the cost of specificity.

What the remediated arm does *not* establish is general recall. It is a measurement of the same eleven classes the baseline located, taken after targeted repair, so 10/11 answers the question "were these gaps closable within this architecture?" and not "how much does this gate see?" The baseline is the arm that carries evidential weight about the system as it stood; the remediated arm carries evidence about the cost of closing what the baseline found, which turned out to be nine rule families in two files.

One gap was deliberately left open. CWE-922 remains undetected on the cloud branch, and no rule was written for it, because none could be written that would be specific. A hardcoded credential in a CDK application appears in general-purpose Python, where a regex cannot reliably distinguish an embedded secret from the many idioms that legitimately *handle* secrets — reading from Secrets Manager, naming a parameter, granting read access. A rule that fired on those would have raised the coverage figure while making the gate less useful, and the withholding is recorded in the catalogue rather than left as an unexplained miss. The on-premises branch faces no such difficulty: its fixture is a playbook, where a credential appears as a literal value in a known field, and a dedicated secret scan names it twice. The converse asymmetry also remains: the learned code-risk component is computed from Python source and has no playbook analogue, so the on-premises path cannot acquire it by reweighting either.

Three limits bound how far these numbers should be read. The fixtures are author-written, so a class counts as covered when the gate names a weakness the author composed to be namable; the coverage figures are an upper bound on what the gate would recover from arbitrary generated infrastructure, and they measure rule-to-CWE alignment rather than field recall. The controls establish that no matcher responds to a compliant artifact, but a control that produces no findings cannot establish that a

matcher will refrain from firing on an *unrelated* finding in a report that contains several. And any gap observed is a property of the particular scanner set assembled in this prototype, not of the process architecture: substituting a scanner with broader on-premises coverage would change the measurement without changing the design.

The substantive result survives all three. Where a class is named on both sides, the identifiers are the same on both sides. The on-premises rules were written to emit the cloud-side categories deliberately, so a weakness carries one name across the boundary and both branches deduplicate and score it through a single code path. Without that shared vocabulary the columns of Table 4.8 would not be comparable at all — and the CWE-732 baseline miss shows what happens when it is absent even once.

4.3.3 RQ3: Unified Risk-Evaluation Capability

Dataset Composition

The corpus was constructed from two classes of Python files. Approximately 800 positive examples were selected from SecurityEval and PyCode-Vul [29], [30]. Vulnerability categories that could not be represented by the configured Bandit and Semgrep features were excluded from the model corpus. The negative reference class contained approximately 800 files sampled from PyCode-Vul [30] and maintained Python projects, including `click`, `rich`, and `typer`.

The negative examples are referred to as *reference secure* files rather than formally verified secure files. Maintained project code can still contain unknown vulnerabilities, and the assigned negative labels therefore represent a practical reference assumption. Similarly, excluding vulnerability categories outside the observable scope of the selected analysers narrows the interpretation of the results. The reported performance applies to the retained, detector-visible subset and is not an estimate of performance over the complete SecurityEval dataset.

The held-out test partition contained 324 samples: 162 negative samples and 162 positive samples. The balanced distribution limited the influence of class frequency on the reported accuracy.

Classification Performance

Table 4.10 presents the aggregate performance of the logistic-regression classifier on the held-out test partition. The classifier achieved an accuracy of 0.9259 and a ROC–AUC of 0.9233. Precision are 0.9480, recall are 0.9012, and F1-score for the positive class are 0.9240.

Table 4.10: Logistic-regression performance on the held-out test partition.

Measure	**Result**
Accuracy	0.9259
Precision	0.9480
Recall	0.9012
F1-score	0.9240
ROC–AUC	0.9233
Test samples	324

The confusion matrix is presented in Table 4.11. Of the 162 negative samples, 146 were classified correctly and 16 were incorrectly classified as insecure. Of the 162 positive samples, 154 were identified correctly and 8 were incorrectly classified as reference secure. The model therefore produced 24 errors across the 324 held-out samples.

Table 4.11: Confusion matrix for the held-out test partition.

	Predicted negative	**Predicted positive**
Actual negative	146	16
Actual positive	8	154

Table 4.12 gives the class-specific results. The negative class obtained precision, recall, and F1-score values of over 0.9, also the positive class obtained over 0.9 for all three measures. The macro and weighted averages

were both approximately 0.93, indicating that performance was similar across the two classes in the held-out partition.

Table 4.12: Class-specific classification results.

Class	**Precision**	**Recall**	**F1-score**	**Support**
Negative (0)	0.95	0.90	0.92	162
Positive (1)	0.91	0.95	0.93	162
Macro average	0.93	0.93	0.93	324
Weighted average	0.93	0.93	0.93	324

The equal positive-class precision and recall resulted from the symmetric number of false-positive and false-negative predictions. From an operational perspective, the two false negatives are particularly relevant because they represent labelled insecure files that would receive a lower learned risk contribution. The two false positives have a different consequence: they may increase the score of reference negative code and cause unnecessary review, but they do not directly permit an insecure file to bypass the learned component.

Analysis of Model Coefficients

Table 4.13 presents the fitted logistic-regression coefficients in descending order. Because the features were standardised before training, coefficient magnitudes provide an indication of their relative influence within this fitted model. A positive coefficient indicates that an increase in the corresponding feature increases the estimated log-odds of the insecure class, with the remaining features held constant.

The feature `bandit_low` received the largest positive coefficient (1.720216), followed by `bandit_conf_high` (1.656566) and `total_bandit` (1.555504). These results indicate that the logistic-regression model relied strongly on Bandit-derived signals, particularly low-severity findings and high-confidence detections, when estimating the probability that a code sample was vulnerable. Among the Semgrep-derived features, `semgrep_high` (0.790939) and

Table 4.13: Feature coefficients learned by the logistic-regression model.

Feature	Coefficient
bandit_low	1.720216
bandit_conf_high	1.656566
total_bandit	1.555504
semgrep_high	0.790939
total_semgrep	0.546920
bandit_conf_medium	0.497096
bandit_high	0.381945
semgrep_medium	0.029059
semgrep_low	0.000000
bandit_medium	−0.359115
bandit_conf_low	−0.366808

`total_semgrep` (0.546920) also contributed positively, whereas `semgrep_medium` (0.029059) had only a marginal influence and `semgrep_low` had no measurable effect. In contrast,`bandit_medium` (−0.359115) and `bandit_conf_low` (−0.366808) received negative coefficients, indicating that higher values of these features slightly reduced the predicted probability of vulnerability after the other variables were taken into account.

Per-File Risk Records

In addition to the aggregate classification measures, the corpus is reprocessed through the trained component to obtain a predicted probability and risk contribution for each file. The resulting machine-readable artifact is archived with the project repository for inspection and reproducibility. Each record contains the sample identifier, dataset source, assigned label, extracted feature values, predicted probability, predicted class, and learned risk contribution. When the complete set of gate inputs is available, the record also contains the component scores, total score, and resulting deployment decision.

Table 4.14 defines the fields retained in this supplementary artifact. The aggregate distribution of per-file probabilities and risk scores is reported after the re-execution records have been finalised.

Table 4.14: Fields retained in the supplementary per-file risk artifact.

Field	**Description**
Sample identifier	Unique identifier or relative path of the evaluated Python file
Source	SecurityEval or the originating reference project
Assigned label	Positive (1) or negative reference (0) class
Feature vector	Bandit and Semgrep counts supplied to the trained model
Model output	Predicted probability, predicted class, and learned risk contribution
Gate output	Component scores, total score, and decision when all gate inputs are available

Table 4.15 presents the observed outcomes of a controlled gate-decision check against the implemented scoring logic. The check evaluates the two decision boundaries directly and confirms that semantically equivalent findings reported by multiple tools are deduplicated into a single retained finding.

Table 4.15: Observed results for the controlled gate-decision evaluation.

Case	**Expected**	**Observed**	**Agreement**
Score below or equal to pass threshold	Pass	Pass at $S = 20$	Yes
Score immediately above pass threshold	Review	Review at $S = 21$	Yes
Score equal to review upper boundary	Review	Review at $S = 80$	Yes
Score above reject threshold	Reject	Reject at $S = 81$	Yes
Duplicate finding reported by multiple tools	Counted once	One deduplicated finding retained	Yes

Chapter 5

Discussion

5.1 Interpretation of Research Questions

5.1.1 RQ1: Decision Conformance and Enforcement Correctness

RQ1 was stated around three properties that do: whether the decision reached is the decision the case warranted, whether that decision constrains what happens next, and whether the evidence behind it is preserved.

On the first, all 106 pre-registered runs produced the decision declared for them in advance, and the confusion matrix over the three bands is exactly diagonal (Table 4.3). The diagonal is the substantive result rather than the aggregate percentage: a gate can score highly on overall agreement while being systematically permissive, and that bias would be hidden inside a single figure. On the second, all six enforcement invariants held on every run to which they applied (Table 4.4) — no *reject* was followed by a deployment, and no *review* was deployed without a matching approval record.

The strength of this evidence should not be overstated in two specific respects. The fixtures were authored by the same work that built the gate, so conformance shows that the implementation satisfies its specification on

Table 5.1: Current evidential status of the research questions.

RQ	Objective	Available evidence	Status
RQ1	Decision conformance, enforcement correctness, evidence completeness	106/106 pre-registered runs conformant; six enforcement invariants held on every applicable run; ten degradation scenarios behaved as predicted	Evaluated
RQ2	Equivalent governance of an on-premises target and a cloud target by the shared risk engine	All seven on-premises checkpoints attained, the last three in a 6-run live-target arm; of eleven catalogued weakness classes, three named on both sides of the boundary at the frozen gate commit and ten after remediation, in a shared finding vocabulary	Evaluated (coverage bounded)
RQ3	Multi-tool risk estimation and deployment decision	Held-out model results available; complete gate-decision analysis incomplete	Partially evaluated

cases whose correct treatment is already known; it does not show that the specification assigns the right band to arbitrary real infrastructure. And the invariants were not all exercised equally often: summary persistence and scanner-status reporting held over all 106 runs, but the approval-record invariant was tested by ten runs and the two rejection invariants by sixteen. Ten clean runs show that the mechanism works when it is invoked; they do not show how often it would fail. What has been established is internal correctness under pre-declared conditions, which is a weaker and more precise claim than "the gate works".

The Gate Fails Open

The most consequential result in the RQ1 data is not the conformance figure but the degradation table. Removing Checkov from a *review*-band cloud change drops its score from 54 to 9 and moves the decision to *pass* (Table 4.5). The gate conformed to its specification here — the scenario declared that outcome in advance — but the specification permits a change to be released precisely because the evidence that would have held it was never collected. The same pattern appears twice more: losing Checkov costs a *reject* fixture 50 of its 114 points, and losing the secret scanner costs the on-premises *reject* fixture 80 of its 155.

This follows directly from the additive form of the scoring function. In $S = \sum_f w(\text{sev}(f)) + c(\Delta\text{cost}) + 5v + m$, a scanner that was never invoked and a scanner that ran and found nothing contribute identically: zero. The score therefore conflates *absence of evidence* with *evidence of absence*, which is the classic failure mode of any evidence-summing control. The system does record which scanners were unavailable — the scanner-status field held for all 106 runs — so an auditor can detect the condition after the fact. But nothing in the decision function consumes that field, so the record is diagnostic rather than protective.

This is a design deficiency rather than an implementation defect, and naming it that way matters: no amount of testing would have surfaced it, because the system behaves exactly as designed. It was surfaced instead by declaring, in advance, what score the remaining evidence *should* support and then observing that the resulting decision was more permissive than the change deserved. Chapter 6 proposes an assurance penalty — a positive contribution for each unavailable scanner, or a fail-closed policy above a coverage threshold — as the remedy.

Gate Overhead

The cloud gate's median duration of 102.50 seconds accounts for roughly seven-tenths of total run wall-clock time, against 7.04 seconds and about a quarter on the on-premises branch. In absolute terms both are acceptable for a pre-deployment check. The concern is the scaling behaviour rather than the current figure: the cloud cost is dominated by Checkov being invoked once per synthesised template file, so it grows with the size of the deployed stack rather than with the size of the change under review. A production deployment with many templates would find the gate, not the cloud provider, to be the bottleneck.

5.1.2 RQ2: Cross-Boundary Enforcement Equivalence

The question asks whether the two sides of the boundary are governed *alike*, which can fail and, as it turned out, did. The workflow-attainment half of the question is answered positively across all seven checkpoints. Syntax validation, security gating and dry run were attained on every attempted on-premises run in the offline campaign, with no failures in any class. The remaining three — apply, post-apply verification, and idempotency — required a real host, and were closed by a live-target arm run against a disposable Multipass virtual machine. That arm supplies a kind of evidence the offline campaign structurally could not: an apply whose resulting state is read back and asserted against independently written expectations, a second apply that reports `changed=0`, and — most importantly for the enforcement claim — three runs that requested deployment of the rejected fixture against a reachable, authenticated host and were stopped at the gate, never reaching the apply stage at all. Enforcement holds at the point of application, not merely in simulation. The arm is small, at three replicates per scenario against one host and one operating system image, so it establishes that the deployment path completes correctly rather than that it is reliable across heterogeneous fleets.

The decision half produced the more interesting finding, and it is not a finding about decisions. Measured against eleven weakness classes derived from an external catalogue, the gate at its frozen commit named seven of them on the cloud branch and five on-premises — but only three on both (Table 4.8). Four classes were visible only in the cloud, two only on-premises, and two on neither. A gate that treats the branches as interchangeable would have been wrong about eight of eleven classes, and nothing in the architecture would have revealed it. Where a class *was* named on both sides, the identifiers matched: an unrestricted firewall rule is reported as `sg_ssh_open` whether it appears in a security group or in an `iptables` task; unrestricted administrative privilege is reported as `iam_wildcard` whether it comes from an IAM policy or a sudoers file. That shared vocabulary is the substantive achievement, because it is what makes the two sides comparable at all. Equal scores would not have been.

One miss shows what happens when the vocabulary breaks down. On the cloud incorrect-permissions fixture the baseline report contained a Checkov finding stating that a KMS key policy contained a wildcard principal — the right weakness, described correctly, in the wrong words. Because the finding carried no category naming the class, it was indistinguishable from silence to the deduplicator, the risk score, the audit record and the operator alike. The same pattern appears more starkly on the plaintext-secret pair, where the cloud report contains five findings about Lambda hygiene and a redundant cfn-lint dependency warning, none of which state that a credential was committed. That fixture still reaches *review*, because the learned code-risk component scores the surrounding Python, so the decision looks correct — but it is correct by accident, and the report an operator would actually read does not mention the secret. Deciding alike is not the same as seeing alike, and only the latter is auditable.

This deserves emphasis because it inverts a common assumption. A shared scoring function and shared thresholds are frequently taken to imply consistent treatment across a hybrid estate. They do not. Consistency of

decision logic is worth little without comparable *detection coverage* feeding it, and the boundary at which coverage diverges is invisible in any architecture diagram — both branches appear to call the same gate, because they do. Nor is such a gap reachable by tuning: a weakness that produces no finding cannot be scored, reviewed, or audited at any threshold, so the remedy is to add the missing detection rather than to reweight what is already present. The remediated arm demonstrates that this remedy was available: nine of the eleven gaps closed with nine rule families added to two files, lifting coverage to ten of eleven in the cloud and eleven of eleven on-premises and reducing the discordant classes from eight to one. That figure should not be read as a recall estimate. It measures whether the gaps the baseline located were closable within this architecture, not how much the gate sees in general; the baseline is the arm that speaks to the latter. The one gap left open was left open deliberately, because a hardcoded credential in a CDK application sits in general-purpose Python where no regex distinguishes an embedded secret from the idioms that legitimately handle one, and a rule that could not be made specific would have raised the number while degrading the tool. The same argument applies in reverse to the learned component, which is computed from Python source and has no playbook analogue.

The evidence supporting this is bounded, though the bound is no longer the size of the class list. Eleven classes drawn from an external catalogue remove the author's discretion over which weaknesses are counted, but not over how each one is written: the fixtures are still hand-authored, so a class counts as covered when the gate names a weakness composed to be namable. The coverage figures are therefore an upper bound measuring rule-to-CWE alignment rather than field recall. The specificity controls narrow this from the other side — both remained silent on all eleven classes in both arms — but a control that produces no findings at all cannot establish that a matcher will refrain from firing on an unrelated finding in a busier report, a failure mode an earlier candidate control exhibited

before it was replaced.

5.1.3 RQ3: Unified Risk-Evaluation Capability

The classifier experiment provides positive but bounded evidence for RQ3. The logistic-regression component achieved an accuracy of 0.9259, an F1-score of 0.9240, and a ROC–AUC of 0.9233 on the held-out partition. The confusion matrix contained 8 false positives and 16 false negatives. Within the filtered corpus, the combined Bandit and Semgrep features therefore provided a useful learned risk signal.

This result does not validate the complete gate. The model was evaluated only on detector-visible Python vulnerabilities, and categories outside the capabilities of the feature extractors were excluded. In addition, the negative files were sampled from maintained projects rather than formally verified as secure. The controlled tests for score aggregation, threshold boundaries, deduplication, and final deployment decisions also remain incomplete. RQ3 is consequently interpreted as partially evaluated: the learned component performed consistently on the selected test set, but the complete multi-component decision mechanism has not yet been fully validated.

5.2 Comparison with Related Approaches

The contribution of the proposed process concerns cloud operations rather than security alone. It coordinates infrastructure definition, assessment, deployment, and monitoring across public and private targets, with security acting as a decision function within the broader workflow.

Traditional infrastructure operations often separate provisioning, system administration, delivery, and security review. This separation can require manual coordination and may delay security feedback. Rajapakse et al. report that conventional security tools are difficult to adapt to rapid

DevOps workflows, while CloudCAMP identifies substantial manual and scripting effort in cloud deployment [31], [32]. The proposed process instead places a common gate before deployment and records the relationship between a change, its evidence, its decision, and its deployed state. However, the present study does not quantify improvements in lead time, operator effort, or incident reduction over a manual baseline.

The NIST Cloud Computing Reference Architecture describes cloud actors, responsibilities, service orchestration, management, security, and privacy, but it does not prescribe a concrete implementation sequence [5]. The proposed process complements this reference view by defining an executable sequence for assessment, approval, deployment, and monitoring. This concreteness reduces generality: the prototype reflects one public-cloud provider and one private-side configuration mechanism, whereas the NIST architecture is vendor-neutral.

Infrastructure as Code improves repeatability and automation, but industrial evidence shows that IaC ecosystems remain fragmented and provide limited support for integration, quality automation, maintenance, and evolution [33]. Research on IaC testing also indicates that executable infrastructure definitions still require explicit verification [34]. The proposed adapter and normalisation layers address heterogeneous scanner output, but they introduce maintenance obligations as tool formats and rules evolve. Compared with CloudCAMP's model-driven deployment abstraction, the proposed process places greater emphasis on risk-based gating and auditability, while offering less platform independence [32].

Continuous-security approaches similarly integrate security activities throughout the delivery lifecycle. Kumar and Goyal's ADOC model proposes automated continuous security using open-source software over cloud infrastructure [35]. The proposed process follows this principle but adds a shared risk decision for public-cloud and private-side changes. Its normalised findings, deduplication, and component breakdown also address the alert fatigue and tool-integration difficulties reported by Rajapakse et

al. [31]. Nevertheless, aggregation may conceal context unless the original findings and component contributions remain available to the reviewer.

Table 5.2: Comparison with related cloud operational approaches.

Approach	**Primary emphasis**	**Relationship to this research**
Traditional operations	Separately coordinated provisioning, administration, and review	The proposed process integrates these transitions, but no quantitative manual baseline has yet been evaluated.
NIST reference architecture [5]	Vendor-neutral actors and cloud components	Provides the architectural context; the proposed process supplies a more concrete but less vendor-neutral operational sequence.
CloudCAMP [32]	Model-driven generation and reduced manual scripting	Provides stronger platform abstraction; this research places greater emphasis on gating, traceability, and monitoring.
Industrial IaC practice [33]	Adoption, integration, and maintenance challenges	Normalisation addresses heterogeneous outputs but creates an adapter layer that must be maintained.
Continuous security [31], [35]	Lifecycle security, tool integration, and triage	This research adds hybrid-cloud decision control and an explainable quantitative score.

5.3 Limitations

The study has seven principal limitations.

1. **Self-authored fixtures and limited scale.** RQ1 and RQ2 were evaluated on a pre-registered campaign of 106 runs whose scenarios and expected outcomes were fixed in advance, which removes the selection freedom of the earlier development sample. It does not remove a more basic dependency: the fixtures were authored by the same work that built the gate, so conformance establishes that the implementation satisfies its own specification on cases whose correct

treatment is already known. It does not establish that the specification assigns the right band to arbitrary real infrastructure, which would require independently authored cases. Several results also rest on very few runs — ten approved-review runs, sixteen rejections, and three replicates for each of the apply, verification and idempotency checkpoints — so they establish that the mechanism works when exercised rather than how often it fails. Detection coverage rests on eleven weakness classes drawn from an external catalogue, which removes the author's discretion over which weaknesses are counted but not over how each one is written. The prototype has additionally not been evaluated with large repositories, concurrent pipelines, or a substantial private-infrastructure fleet.

2. **Narrow deployment-side evidence.** Apply, post-apply verification, and repeated-run idempotency were measured, but only in the live-target arm: three replicates of one scenario against a single disposable virtual machine running one operating system image. What that supports is that the deployment path completes correctly and that a *reject* decision blocks application against a real host. It does not support claims about behaviour under host heterogeneity, concurrent targets, partial failures, or hosts whose prior state differs from a fresh image — all of which are exactly the conditions under which convergence tooling tends to misbehave.

3. **Restricted implementation scope.** The learned component is limited to Python and uses features from Bandit and Semgrep. The wider prototype relies on a small fixed set of adapters and is centred on AWS CDK, CloudFormation, AWS operational services, and Ansible. The findings cannot be assumed to transfer to other languages, cloud providers, configuration systems, or vulnerability classes.

4. **Dataset and model validity.** The held-out result contains only

31 samples and was obtained from one train–test split. Vulnerability categories outside the selected analysers' detection scope were excluded, and maintained project files were treated as negative examples without formal security verification. These choices introduce selection bias, possible label noise, and uncertainty about performance on external projects.

5. **Risk assumptions and the fail-open score.** The severity weights, cost bands, compliance contribution, model contribution, and decision thresholds have not been calibrated using expert judgement or historical incidents. Exposure, asset criticality, data sensitivity, and compensating controls are represented only indirectly. More seriously, the campaign confirmed that the additive score *fails open*: an unavailable scanner and a scanner that found nothing contribute identically, so removing Checkov moved a *review*-band change to *pass* (Table 4.5). The condition is recorded in the scanner-status field but no part of the decision function acts on it.

6. **Unequal detection coverage across the boundary.** The two branches share a decision function but not a scanner set. At the frozen gate commit the branches agreed on only three of eleven catalogued weakness classes; adding nine rule families raised this to ten, but the cloud path still names no committed credential, detecting one only as a side effect of the learned code-risk model scoring the surrounding Python. That model is in turn unavailable to the on-premises path, which has no source-code analogue. Each side therefore retains a blind spot the other does not. Coverage is also established over hand-authored fixtures, so the figures bound rule-to-CWE alignment rather than recall on real infrastructure, and the specificity controls — silent on all eleven classes in both arms — cannot exclude a matcher firing on an unrelated finding in a busier report.

7. **Limited comparative and operational measures.** The study does not yet compare the proposed process with manual cloud operations or IaC deployment without the unified gate. It also does not measure operator effort, deployment lead time, change-failure rate, remediation time, false-alert handling, or long-term incident reduction. The current findings therefore establish limited component feasibility rather than overall operational superiority.

5.4 Directions for Improvement

The first priority is to close the fail-open condition, since it is the one limitation that weakens the control while the system reports success. Making the decision function consume the scanner-status field it already records — either as an assurance penalty proportional to missing coverage, or as a fail-closed policy that forces *review* below a coverage threshold — would convert a diagnostic record into an enforced property, and the existing degradation scenarios already provide the test cases to verify it.

The second is to close the remaining coverage gap at its source, on both sides. The cloud branch needs a secret scan that can separate an embedded credential from the idioms that legitimately handle one — entropy and provider-format detection rather than the keyword regex the on-premises branch runs — so that a committed credential is named in the report rather than absorbed into a learned probability; this was the one gap deliberately left open, because nothing specific enough could be built within the existing regex design. The on-premises branch needs either an equivalent learned signal for configuration artifacts or an explicit statement that no such component applies, so that a reader can tell a missing component from a component that found nothing. Both are coverage repairs, and neither requires the two branches to produce the same score. The measurement itself should then be strengthened by replacing the hand-authored fixtures with independently written ones, since the present figures

bound rule-to-catalogue alignment rather than recall, and by widening the class list beyond the eleven that admit a matched expression on both sides of the boundary.

The evaluation itself should then be broadened on the deployment side. The live-target arm closes the apply, post-apply verification, and idempotency checkpoints, but against a single fresh virtual machine; the informative extension is to repeat it against hosts with divergent prior state, several targets concurrently, and induced partial failures, so that the failure-class attribution has something other than `none` to classify. Independently authored fixtures would address the self-authorship dependency. Subsequent evaluation should increase repository size, infrastructure scale, and pipeline concurrency while measuring execution time, resource consumption, storage growth, and cloud-service cost — the measured gate overhead, at roughly seven-tenths of cloud run time and scaling with template count, indicates where that cost will concentrate. A controlled comparison among manual operations, IaC automation without the gate, and the proposed process would provide evidence regarding operator effort, lead time, policy violations, and remediation time.

The code-risk model should be evaluated using a larger multi-project corpus, project-aware data partitioning, repeated cross-validation, and an external holdout set. Vulnerabilities outside the current analyser scope should be retained as a separate challenge set rather than removed entirely. Additional language-specific models and features covering dependency risk, resource exposure, identity policy, data sensitivity, and runtime behaviour would broaden the assessment scope.

The unified score should be calibrated using expert-labelled changes, historical incidents, and organisational risk preferences. Sensitivity and ablation analyses would show how individual components affect decisions and whether the learned signal adds value beyond severity-only or scanner-only baselines. Missing assessment components should produce an assurance penalty or configurable fail-closed behaviour so that insufficient evidence

is not interpreted as low risk.

Finally, the provider and adapter interfaces should be extended to additional cloud and private-infrastructure platforms. Evaluation with system administrators, developers, and security practitioners would determine whether the component breakdown improves decision-making or obscures important detail. Versioned policies, role-separated approvals, signed audit records, and longitudinal deployment would further strengthen the governance and practical applicability of the process.

Chapter 6

Conclusion

6.1 Research Summary

Hybrid cloud environments combine the flexibility of public-cloud services with the control of private infrastructure, but this combination also increases operational complexity. Infrastructure provisioning, system administration, software delivery, and security assessment are frequently performed through separate tools and workflows. This separation makes it difficult to maintain consistent policies, coordinate changes, and preserve traceability across heterogeneous infrastructure. The central objective of this research was therefore to design and implement a SysSecOps operational process that integrates these activities into a unified workflow for hybrid cloud environments.

The research first examined the operational and security challenges associated with hybrid cloud management. The analysis identified fragmented tooling, inconsistent assessment outputs, delayed security feedback, limited coordination between public and private infrastructure, and insufficient empirical evaluation of integrated operational processes. These challenges motivated an approach in which security is not treated as a separate final activity, but as a continuous decision function within infrastructure generation, assessment, deployment, and monitoring.

The proposed process organises the operational lifecycle into a sequence

that begins with an infrastructure request or existing project. Infrastructure artifacts are generated or accepted, validated, and transformed into the concrete representations used for deployment. Multiple assessment components then examine the artifacts from complementary perspectives. Their outputs are normalised, deduplicated, and converted into a unified risk score. The resulting pass, review, or reject decision controls whether deployment is permitted. After an accepted deployment, monitoring and audit records connect the operational state of the target to the evidence and decision that preceded it.

The process was implemented as a Python-based prototype. Public-cloud infrastructure was represented through AWS CDK and CloudFormation, while private infrastructure was represented through Ansible-managed Linux nodes. The prototype incorporated infrastructure validation, security scanning, cost information, live compliance state, a learned code-risk signal, deployment governance, and operational monitoring. Although these technologies provided the implementation environment, the principal research contribution is the operational sequence and decision structure rather than the use of a particular tool.

The risk-scoring mechanism combines deduplicated severity findings, estimated cost, live-compliance observations, and a machine-learning code-risk contribution. This additive structure was selected to preserve transparency: each component contributes an explicit number of points, and the total is mapped to a defined operational action. The gate report retains the original findings, component execution states, score breakdown, and final decision so that an operator can trace the decision to its supporting evidence.

The evaluation was structured around three research questions. RQ1 concerns whether the pipeline reaches the decision each change warrants and whether that decision constrains what follows. RQ2 concerns whether the two sides of the hybrid boundary are governed alike by the shared risk engine. RQ3 concerns the use of multiple analysis signals and logistic

regression within the unified risk mechanism. RQ1 and RQ2 were evaluated through a pre-registered campaign of 106 runs across 30 scenarios whose expected outcomes were fixed in a version-controlled specification before execution. All 106 runs produced the declared decision, the confusion matrix over the three bands was exactly diagonal, and six enforcement invariants — including that no rejection was ever deployed and no review was deployed without an approval record — held on every applicable run. On the on-premises path, syntax validation, security gating and dry run were attained on all 59 offline runs, and a separate live-target arm of 6 runs against a disposable virtual machine closed the remaining three checkpoints: apply succeeded, post-apply verification asserted the resulting host state against independently written expectations, a repeated apply reported no further change, and the runs that requested deployment of a rejected fixture were stopped at the gate without ever reaching the host. RQ3 is partially evaluated through the held-out performance of the learned code-risk component.

Two findings emerged that the pipeline's own success criteria would not have surfaced. First, the additive score *fails open*: removing one scanner moved a *review*-band change to *pass*, because an unavailable scanner and a scanner that found nothing contribute identically. Second, detection coverage is unequal across the hybrid boundary. Measured against eleven weakness classes derived from an external catalogue, the gate at its frozen commit named seven on the cloud branch and five on-premises, but only three on both: four were visible only in the cloud, two only on-premises, and two on neither. Writing nine rule families closed all but one of those gaps, raising the branches to ten and eleven of eleven respectively. A gap of that kind is not reachable by adjusting weights or thresholds — a weakness that produces no finding cannot be scored, reviewed, or audited at any setting — and it was visible only because the same weakness was expressed on both sides and the two reports were compared.

The classifier evaluation used a filtered corpus containing vulnerable

Python examples from SecurityEval and reference negative files from maintained Python projects. On the 31-file held-out partition, the logistic-regression model achieved an accuracy of 0.8710, a precision of 0.8667, a recall of 0.8667, an F1-score of 0.8667, and a ROC–AUC of 0.8792. The confusion matrix contained two false positives and two false negatives. These findings indicate that the combined Bandit and Semgrep features provide a useful learned signal within the detector-visible evaluation scope. They do not, however, establish complete vulnerability detection or full validation of the unified deployment gate.

6.2 Research Contributions

This research makes three principal contributions.

First, it proposes a SysSecOps operational process for hybrid cloud environments. The process integrates infrastructure management, software delivery, continuous assessment, deployment governance, and operational monitoring. Its main distinction is the use of a common decision path across public and private targets, reducing the separation between system administration, development, and security activities. The process also defines the evidence that must be retained at each stage, supporting traceability from an infrastructure change to its deployed operational state.

Second, the research develops a unified risk-evaluation mechanism for heterogeneous assessment outputs. Scanner findings are converted into a common representation and deduplicated before they contribute to the score. Severity is combined with operational context in the form of cost, live compliance, and a learned code-risk probability. This mechanism transforms several independent outputs into a single operational decision while retaining a component-level explanation. The preliminary classifier results demonstrate the feasibility of incorporating a learned multi-tool signal, although the complete decision mechanism requires further validation.

Third, the research provides a working prototype and a reproducible

evaluation framework. The prototype demonstrates how generation, infrastructure automation, assessment adapters, scoring, approval control, deployment, and monitoring can be connected through shared artifacts and machine-readable reports. The evaluation framework defines separate criteria for pipeline completion, private-target integration, classifier performance, score aggregation, decision boundaries, and deduplication. This separation prevents a successful command execution or an intentional security rejection from being misinterpreted as evidence for a broader research claim.

Together, these contributions shift the focus from isolated security-tool execution to the coordination of the complete cloud operational lifecycle. The proposed process does not replace established cloud reference architectures, IaC practices, or continuous- security approaches. Instead, it connects these concerns through an explicit sequence of artifacts, decisions, and operational records tailored to hybrid infrastructure.

6.3 Future Work

The immediate priority is to remove the fail-open condition, since it is the one deficiency that weakens the control while the system reports success. The gate already records which scanners were unavailable; the decision function should consume that record, either as an assurance penalty proportional to missing coverage or as a configurable fail-closed policy that forces *review* when coverage falls below a threshold. The existing degradation scenarios provide ready-made test cases for verifying such a change, since each declares in advance what the remaining evidence ought to support.

The second priority is to close the residual coverage gap at its source, on both sides of the boundary. The cloud path should run a secret scan capable of distinguishing a credential embedded in a CDK application from the idioms that legitimately handle one — entropy and provider-format

detection rather than the keyword regex the private path uses — so that a committed credential appears as a named finding rather than as an increment to a learned probability that an operator cannot interpret. This was the one gap left deliberately open, because no rule specific enough to be worth having could be written within the existing regex-based design. The private path should either gain an equivalent learned signal over configuration artifacts or declare explicitly that no such component applies, so that a missing component is distinguishable from a component that found nothing. Both are coverage repairs and neither requires the two branches to produce equal scores, which they should not be expected to do: a wildcard IAM policy and a sudoers entry are not the same object and do not admit the same number of legitimate findings. Beyond closing that gap, the coverage measurement itself should be strengthened by replacing the hand-authored fixtures with independently written ones, since the present figures bound rule-to-catalogue alignment rather than recall, and by widening the class list beyond the eleven that admit a matched expression on both sides of the boundary.

The deployment-side evaluation should then be broadened. The live-target arm establishes that apply, post-apply verification and repeated-run idempotency succeed against a fresh host, but every recorded run fell into the failure class `none`, which means the attribution machinery has not yet been exercised. Repeating the arm against hosts with divergent prior state, against several targets concurrently, and under induced partial failures would test whether failures are classified and surfaced as designed. Independently authored fixtures would address the self-authorship dependency in the present results.

The evaluation should subsequently be extended to larger repositories, more infrastructure resources, concurrent pipelines, and multiple private targets. Relevant measures include synthesis time, scan duration, total pipeline latency, resource consumption, cloud-service cost, storage growth, and recovery from interrupted or partial execution. The measured gate

overhead indicates where that cost will concentrate: the cloud gate consumed roughly seven-tenths of run wall-clock time and scales with the number of synthesised templates rather than with the size of the change, so incremental scanning of changed templates is a natural optimisation. A controlled comparison with manual cloud operations and IaC automation without the unified gate would determine whether the process improves lead time, operator effort, policy compliance, traceability, and remediation time.

The learned code-risk component should be validated using a larger multi-project corpus, project-aware data partitioning, repeated cross-validation, and an independent external test set. Vulnerabilities that cannot be detected by the current static analysers should be retained as a challenge set to quantify the model's blind spots. Support for other programming and IaC languages requires new labelled datasets, language-specific feature extractors, and independently trained models.

The scoring mechanism should be calibrated using expert assessments, historical changes, incident records, and organisational risk preferences. Sensitivity and ablation analyses can determine the effect of each score component and whether the learned contribution provides value beyond severity-only or scanner-only baselines. Exposure, asset criticality, data sensitivity, dependency risk, and compensating controls should be incorporated where reliable evidence is available. Missing assessment components should produce an assurance penalty or a configurable fail-closed outcome for high-risk environments.

Finally, the prototype should be extended to additional public-cloud providers, private-cloud platforms, and orchestration technologies. Evaluation with system administrators, developers, and security practitioners would determine whether the score breakdown assists decision-making and remediation. Versioned policy bundles, role-separated approval, signed audit records, and longitudinal deployment would further improve governance and support practical adoption.

6.4 Concluding Remarks

This research presents a structured approach to coordinating infrastructure management, software delivery, security assessment, and operational monitoring in a hybrid cloud environment. Its central proposition is that security should function as an explicit, auditable operational decision rather than as an isolated activity after deployment. The proposed process, unified scoring mechanism, and prototype provide a foundation for this integration. The pre-registered campaign showed that the implemented gate decides and enforces as specified; it also showed that the specification itself permits a change to be released when evidence is missing, and that a shared decision function does not by itself produce equal treatment across a hybrid boundary when detection coverage differs. Those two findings are as much a result of this work as the conformance figures, and they define the agenda for further validation. With broader evaluation, calibration, and platform coverage, the approach can provide a basis for more consistent and traceable SysSecOps practice across heterogeneous cloud infrastructure.

Reference

[1] M. Armbrust et al., "Above the clouds: A berkeley view of cloud computing," Tech. Rep. UCB/EECS-2009-28, Feb. 2009. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2009/EECS-2009-28.html

[2] B. Varghese and R. Buyya, "Next generation cloud computing: New trends and research directions," *Future Generation Computer Systems*, vol. 79, pp. 849–861, 2022.

[3] Flexera, *2026 state of the cloud report*, https://www.flexera.com/blog/finops/flexera-2026-state-of-the-cloud-report-the-convergence-of-cloud-and-value/, Accessed: 2026-06-05, 2026.

[4] P. Mell and T. Grance, "The nist definition of cloud computing," National Institute of Standards and Technology, Tech. Rep. Special Publication 800-145, 2011. DOI: 10.6028/NIST.SP.800-145 [Online]. Available: https://doi.org/10.6028/NIST.SP.800-145

[5] F. Liu et al., "Nist cloud computing reference architecture," National Institute of Standards and Technology, Tech. Rep. Special Publication 500-292, 2011. DOI: 10.6028/NIST.SP.500-292 [Online]. Available: https://doi.org/10.6028/NIST.SP.500-292

[6] M. Armbrust et al., "A view of cloud computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010. DOI: 10.1145/1721654.1721672

[7] Flexera, *2025 state of the cloud report*, https://info.flexera.com/CM-REPORT-State-of-the-Cloud, Accessed: 2026-07-17, 2025.

[8] HashiCorp, *2025 state of cloud strategy survey*, https://www.hashicorp.com/state-of-the-cloud, Accessed: 2026-07-17, 2025.

[9] K. Dodson, M. Souppaya, and K. Scarfone, "Secure software development framework (ssdf) version 1.1," National Institute of Standards and Technology, Tech. Rep. Special Publication 800-218, 2022. DOI: 10.6028/NIST.SP.800-218

[10] H. Myrbakken and R. Colomo-Palacios, "Devsecops: A multivocal literature review," in *International Conference on Software Process Improvement and Capability Determination (SPICE)*, ser. Lecture Notes in Business Information Processing, vol. 290, Springer, 2017, pp. 17–29. DOI: 10.1007/978-3-319-67383-7_2

[11] R. N. Rajapakse, M. Zahedi, M. A. Babar, and H. Shen, "Challenges and solutions when adopting devsecops: A systematic review," *ACM Computing Surveys*, vol. 55, no. 11, pp. 1–45, 2023. DOI: 10.1145/3579635

[12] X. Zhao, T. Clear, and R. Lal, "Identifying the primary dimensions of devsecops: A multi-vocal literature review," *Journal of Systems and Software*, vol. 214, p. 112 063, 2024, ISSN: 0164-1212. DOI: 10.1016/j.jss.2024.112063

[13] L. Bass, I. Weber, and L. Zhu, *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional, 2015, ISBN: 9780134049847.

[14] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010, ISBN: 9780321601919.

[15] OWASP Foundation, *OWASP Software Assurance Maturity Model (SAMM) Version 2.1*, https://owaspsamm.org/, 2024.

[16] P. Mell, K. Scarfone, and S. Romanosky, "Guide to enterprise patch management planning: Preventive maintenance for technology," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-40 Revision 4, 2022. DOI: 10.6028/NIST.SP.800-40r4

[17] Forum of Incident Response and Security Teams (FIRST), *Common Vulnerability Scoring System Version 4.0: Specification Document*, https://www.first.org/cvss/v4.0/specification-document, 2023.

[18] C. Maplestone et al., *Stakeholder-Specific Vulnerability Categorization (SSVC) Version 2.0*, https://certcc.github.io/SSVC/, 2023.

[19] M. Bozorgi, L. K. Saul, S. Savage, and G. M. Voelker, "Beyond heuristics: Learning to classify vulnerabilities and predict exploits," in *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2010, pp. 105–114. DOI: 10.1145/1835804.1835821

[20] Y. Shin and L. Williams, "Can traditional fault prediction models be used for vulnerability prediction?" *Empirical Software Engineering*, vol. 18, no. 1, pp. 25–59, 2013. DOI: 10.1007/s10664-011-9193-7

[21] F. Rahman, D. Posnett, A. Hindle, and P. Devanbu, "Bugcache for inspections: Hit or miss?" *IEEE Transactions on Software Engineering*, 2019.

[22] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[23] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 1877–1901.

[24] M. Chen, J. Tworek, H. Jun, Q. Yuan, et al., *Evaluating large language models trained on code*, 2021. arXiv: 2107.03374 [cs.LG].

[25] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2022, pp. 754–768. DOI: 10.1109/SP46214.2022.9833571

[26] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023, pp. 2785–2799. DOI: 10.1145/3576915.3623157

[27] MITRE Corporation, *CWE VIEW: Architectural Concepts (CWE-1008)*, https://cwe.mitre.org/data/definitions/1008.html, Common Weakness Enumeration, version 4.20, released 2026-04-30. Accessed: 2026-08-05, 2026.

[28] M. L. Siddiq and J. C. S. Santos, "SecurityEval dataset: Mining vulnerability examples to evaluate machine learning-based code generation techniques," in *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security*, 2022. DOI: 10.1145/3549035.3561184

[29] M. L. Siddiq and J. C. S. Santos, "Securityeval dataset: Mining vulnerability examples to evaluate machine learning-based code generation techniques," in *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security (MSR4P&S22)*, 2022. DOI: 10.1145/3549035.3561184

[30] T. Karim, M. S. Akter, and A. Cuzzocrea, "A benchmark dataset for code-level vulnerability detection and analysis," in *2025 IEEE International Conference on Big Data (BigData)*, IEEE, 2025, pp. 4237–4246.

[31] R. N. Rajapakse, M. Zahedi, and M. A. Babar, "An empirical analysis of practitioners' perspectives on security tool integration into

DevOps," in *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 2021, pp. 1–12. DOI: 10.1145/3475716.3475776

[32] A. Bhattacharjee, Y. Barve, A. Gokhale, and T. Kuroda, *Cloud-CAMP: Automating cloud services deployment and management*, 2019. DOI: 10.48550/arXiv.1904.02184 arXiv: 1904.02184 [cs.SE].

[33] M. Guerriero, M. Garriga, D. A. Tamburri, and F. Palomba, "Adoption, support, and challenges of infrastructure-as-code: Insights from industry," in *2019 IEEE International Conference on Software Maintenance and Evolution*, 2019, pp. 580–589. DOI: 10.1109/ICSME.2019.00092

[34] M. M. Hasan, F. A. Bhuiyan, and A. Rahman, "Testing practices for infrastructure as code," in *Proceedings of the 1st ACM SIGSOFT International Workshop on Languages and Tools for Next-Generation Testing*, 2020. DOI: 10.1145/3416504.3424334

[35] R. Kumar and R. Goyal, "Modeling continuous security: A conceptual model for automated DevSecOps using open-source software over cloud," *Computers & Security*, vol. 97, p. 101 967, 2020. DOI: 10.1016/j.cose.2020.101967